Algorithm

Lecture resume BA4 – EPFL
lecture of Ola Svensson

Liam Vuilleumier

2026

Contents

1 Lecture 1 : Introduction to Algorithm

1.1 What is an algorithm?

An **algorithm** is a well-defined computational procedure that takes one or more values as **input** and produces one or more values as **output**: it is a sequence of steps transforming input into output.

We distinguish an algorithm from a **data structure**, which is a method for storing and organizing information to facilitate its access and modification.

1.2 Introductory example: arithmetic sum

Problem: calculate $S = \sum_{i=1}^{n} i$.

Naive approach

Loop: add $1 + 2 + \cdots + n$

n additions $\Rightarrow$ $\Theta(n)$

vs.

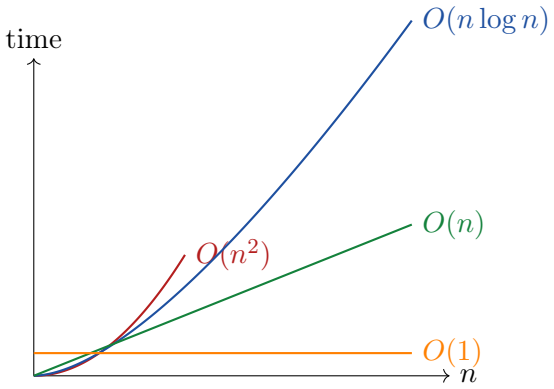
Gauss's approach

Closed-form formula: $\frac{n(n+1)}{2}$

3 operations $\Rightarrow$ $\Theta(1)$

1.3 Why does efficiency matter?

A *slow* computer running a good algorithm will always eventually beat a *fast* computer running a bad algorithm, as soon as the input size n becomes large enough.



Conclusion

The **asymptotic performance** (behavior as $n \to \infty$) is more decisive than the multiplicative constants related to the hardware.

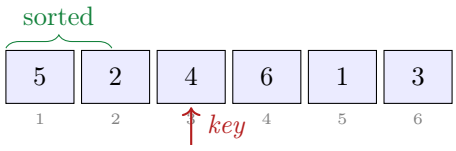
2 Lecture 2 : sort and analyse complexity

2.1 The Sorting Problem

Input: a sequence of n numbers $\langle a_1, a_2, \ldots, a_n \rangle$.
Output: a permutation $\langle a'_1, a'_2, \ldots, a'_n \rangle$ such that $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

2.2 Insertion Sort

The idea is similar to sorting a hand of cards: we go through the elements one by one and insert each new element into its correct position among those already sorted.



Algorithm 1 Insertion-Sort(A)

```
1: for j = 2 to A.length do
2:     key ← A[j]                          // element to be inserted
3:     i ← j − 1
4:     while i > 0 and A[i] > key do
5:         A[i + 1] ← A[i]                 // shift to the right
6:         i ← i − 1
7:     end while
8:     A[i + 1] ← key                      // insert in the correct place
9: end for
```

2.3 Proof of Correctness: Loop Invariant

To prove that an iterative algorithm is correct, we use a **loop invariant**, a property verified at three key moments:

1. **Initialization**: true before the first iteration.
2. **Maintenance**: if true before an iteration, it remains true before the next one.
3. **Termination**: at the end of the loop, the invariant proves correctness.

Invariant for Insertion Sort: at the start of each iteration j, the subarray $A[1 \dots j-1]$ contains the elements originally in $A[1 \dots j-1]$, *but sorted.*

2.4 Complexity Analysis (RAM model)

We use the **Random-Access Machine (RAM)** model: each simple instruction (arithmetic, memory access, test) takes a constant time.

Case	Condition	Complexity
Best case	Array already sorted (while never executes)	$\Theta(n)$
Worst case	Array sorted in reverse order	$\Theta(n^2)$

Worst-case calculation:

$$T(n) = \sum_{j=2}^{n} (j-1) = \frac{n(n-1)}{2} = \Theta(n^2)$$

Why focus on the worst case?

It provides an **upper bound guarantee** on the execution time, valid for any possible input. This is the standard reference in algorithm analysis.

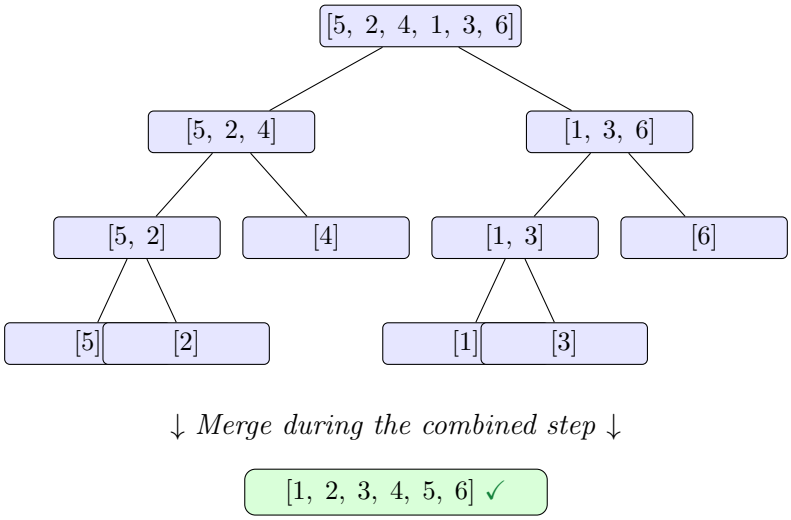
3 Lecture 3 : divide to conquer & Fusion sort

3.1 The Divide and Conquer Paradigm

This recursive approach breaks down any problem into three steps:

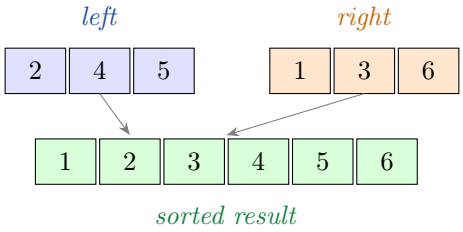
1. **Divide**: break the problem into smaller subproblems of the same nature.
2. **Conquer**: solve each subproblem recursively (base case: direct solution if the problem is small enough).
3. **Combine**: merge the partial solutions into a global solution.

3.2 Merge Sort



3.2.1 The Merge Procedure

The key step: merge two already sorted subarrays. The heads of both lists are compared and the smallest element is placed in the output. This operation takes **linear** time $\Theta(n)$.



3.3 Analysis of Merge Sort

Let $T(n)$ be the time to sort n elements:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 2\,T(n/2) + \Theta(n) & \text{if } n > 1. \end{cases}$$

Recursion tree: at each level k, there are 2^k subproblems of size $n/2^k$, each contributing $\Theta(n/2^k)$. The total cost per level is $\Theta(n)$, and there are $\log_2 n$ levels, therefore:

$$\boxed{T(n) = \Theta(n \log n)}$$

Comparison: Insertion Sort vs. Merge Sort

Algorithm	Worst case	In place?
Insertion Sort	$\Theta(n^2)$	✓
Merge Sort	$\Theta(n \log n)$	×

4 Lecture 4 : Recurence analysis & Maximum sub-array

4.1 Analysis of Divide-and-Conquer Algorithms

The running time $T(n)$ of a divide-and-conquer algorithm is described by a **recurrence** of the form:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq c, \\ a\,T(n/b) + D(n) + C(n) & \text{otherwise,} \end{cases}$$

where $D(n)$ is the cost of division and $C(n)$ is the cost of combination. For merge sort, this gives $T(n) = 2T(n/2) + \Theta(n)$, which results in $T(n) = \Theta(n \log n)$.

4.2 Solving Recurrences

There are three main methods.

4.2.1 Substitution Method

We **guess** the form of the solution, then use mathematical induction to prove the constants.

Example: $T(n) = T(n/4 + 1) + T(3n/4 - 1) + \Theta(1)$. *Let us show that* $T(n) = O(n)$, *i.e.,* $T(n) \leq cn - d$ *for constants* $c, d > 0$.

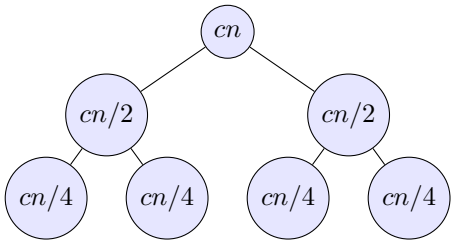
Proof. By inductive hypothesis, the recursive calls satisfy the bound. With $\Theta(1) \leq k$:

$$\begin{aligned} T(n) &\leq c\left(\tfrac{n}{4} + 1\right) - d + c\left(\tfrac{3n}{4} - 1\right) - d + k \\ &= cn - 2d + k. \end{aligned}$$

It suffices to choose $d \geq k$ to have $T(n) \leq cn - d$. □ □

4.2.2 Recursion Trees

We **visualize** the cost at each level of subproblems. Useful for generating a conjecture to be verified later by substitution.



Level 0: cn
Level 1: $cn/2 + cn/2 = cn$
Level 2: $4 \times cn/4 = cn$
$\vdots$
Total: $cn \times \log_2 n = \Theta(n \log n)$

4.2.3 Master Theorem

A "cookbook" for recurrences of the form $T(n) = aT(n/b) + f(n)$ (with $a \geq 1$, $b > 1$). We compare $f(n)$ and $n^{\log_b a}$:

Case	Condition on $f(n)$	Result	Intuition
1	$f(n) = O\big(n^{\log_b a - \varepsilon}\big)$, $\varepsilon > 0$	$T(n) = \Theta\big(n^{\log_b a}\big)$	Leaves dominate
2	$f(n) = \Theta\big(n^{\log_b a}\big)$	$T(n) = \Theta\big(n^{\log_b a} \log n\big)$	Cost is evenly distributed
3	$f(n) = \Omega\big(n^{\log_b a + \varepsilon}\big)$, $\varepsilon > 0$	$T(n) = \Theta(f(n))$	Root dominates

Examples of application:

- Merge Sort: $T(n) = 2T(n/2) + \Theta(n)$. Here $a = 2$, $b = 2$, $n^{\log_2 2} = n$. Case 2 $\Rightarrow T(n) = \Theta(n \log n)$.
- Binary Search: $T(n) = T(n/2) + \Theta(1)$. Here $n^{\log_2 1} = 1$. Case 2 $\Rightarrow T(n) = \Theta(\log n)$.
- Strassen: $T(n) = 7T(n/2) + \Theta(n^2)$. $n^{\log_2 7} \approx n^{2.81} \succ n^2$. Case 1 $\Rightarrow T(n) = \Theta(n^{\log_2 7})$.

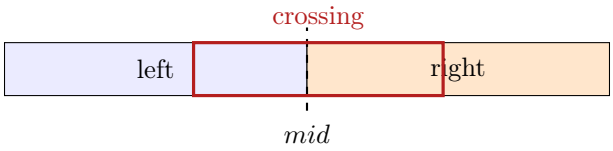
4.3 Maximum Subarray Problem

Input: an array $A[low \ldots high]$ of n integers (some negative).
Output: the indices i, j and the maximum sum $\sum_{k=i}^{j} A[k]$.

4.3.1 Divide-and-Conquer Approach

We divide A into two halves $A[low \ldots mid]$ and $A[mid{+}1 \ldots high]$. The maximum subarray is either:

1. Entirely in the **left** half,
2. Entirely in the **right** half,
3. **Crossing the midpoint** (partly in the left of mid, partly in the right).



The FIND-MAX-CROSSING-SUBARRAY function scans from *mid* toward the left then toward the right in $\Theta(n)$. The global recurrence is:

$$T(n) = 2T(n/2) + \Theta(n) \;\Rightarrow\; \boxed{T(n) = \Theta(n \log n)}$$

5 Lecture 5 : matrice multiplication & heap

5.1 Matrix Multiplication

Problem: given two $n \times n$ matrices A and B, compute $C = A \cdot B$ where $c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$.

Algorithm	Recurrence	Complexity	Remark
Naive (3 nested loops)	—	$\Theta(n^3)$	Reference
Simple D&C (8 subproblems)	$T(n) = 8T(n/2) + \Theta(n^2)$	$\Theta(n^3)$	No improvement
Strassen	$T(n) = 7T(n/2) + \Theta(n^2)$	$\Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$	Better!

Strassen's idea: use clever algebraic combinations to perform only **7 multiplications** of submatrices instead of 8, at the cost of a few additional (less expensive) additions.

Applying the Master Theorem to Strassen

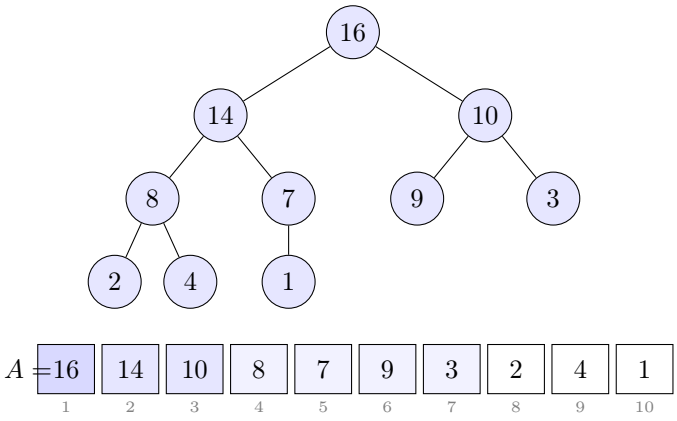
$a = 7$, $b = 2$, $f(n) = \Theta(n^2)$, $n^{\log_2 7} \approx n^{2.81}$.
Since $f(n) = O(n^{\log_2 7 - \varepsilon})$ (case 1): $T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$.

5.2 Heaps and Heapsort

Heapsort combines the advantages of merge sort ($O(n \log n)$) and insertion sort (*in-place* sorting).

5.2.1 Binary Heap Structure

A **max-heap** is a *nearly complete* binary tree stored in an array.



For a node at index i:

$$\text{PARENT}(i) = \lfloor i/2 \rfloor, \quad \text{LEFT}(i) = 2i, \quad \text{RIGHT}(i) = 2i + 1.$$

Max-heap property: $A[\text{PARENT}(i)] \geq A[i]$ for all $i \neq$ root. The maximum is always at the root $A[1]$.

5.2.2 Key Operations

MAX-HEAPIFY(A, i)**.** Restores the max-heap property for the subtree rooted at i, assuming that the left and right subtrees are already max-heaps. The element "drifts" downward. Complexity: $O(\log n)$ (heap height).

BUILD-MAX-HEAP(A)**.** Builds a max-heap from an arbitrary array by calling MAX-HEAPIFY on all non-leaf nodes (from $\lfloor n/2 \rfloor$ down to 1).

Algorithm 2 Build-Max-Heap(A)

```
1: A.heap_size ← A.length
2: for i ← ⌊A.length/2⌋ downto 1 do
3:     MAX-HEAPIFY(A, i)
4: end for
```

Complexity of Build-Max-Heap

Although we call a $O(\log n)$ function $O(n)$ times, a careful analysis (the majority of nodes have a low height) yields:

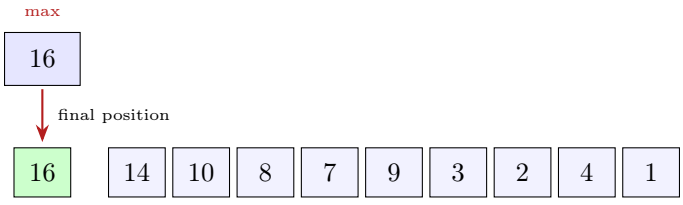
$$\boxed{T_{\text{build}} = \Theta(n)}$$

6 Lecture 6 : Heapsort & priority files

6.1 The Heapsort Algorithm

Heapsort sorts an array *in-place* in $O(n \log n)$ in the worst case.

1. **Build** a max-heap from the input array: $O(n)$.
2. **Repeat** $n - 1$ **times:**
   a. Exchange the root (maximum) $A[1]$ with the last element $A[heap\_size]$.
   b. Reduce $heap\_size$ by 1 (the exchanged element is in its final position).
   c. Call MAX-HEAPIFY$(A, 1)$ to restore the property: $O(\log n)$.



Complexity 6.1: Heapsort

- BUILD-MAX-HEAP: $O(n)$.
- $n - 1$ calls to MAX-HEAPIFY: $O(\log n)$ each.
- **Total:** $O(n \log n)$ **in all cases.**

6.2 Priority Queues

A **(max) priority queue** maintains a dynamic set of elements with keys, and supports:

Operation	Description	Time
MAXIMUM(S)	Returns the element with the maximum key	$O(1)$
EXTRACT-MAX(S)	Removes and returns the maximum	$O(\log n)$
INCREASE-KEY(S, x, k)	Increases the key of x to k	$O(\log n)$
INSERT(S, x)	Inserts x into S	$O(\log n)$

Implementation: the binary heap implements all these operations efficiently.

EXTRACT-MAX.

1. Save $A[1]$ (the maximum).
2. Put $A[heap\_size]$ at the root, reduce $heap\_size$.
3. Call MAX-HEAPIFY($A, 1$).

INCREASE-KEY(A, i, k)**.** After updating $A[i] \leftarrow k$, move up the tree as long as the max-heap property is violated (the element "floats" upward).

Application: Priority Queue

Used in graph algorithms (Dijkstra, Prim) to efficiently extract the next vertex to be processed.

7 Lecture 7 : elementary data structure

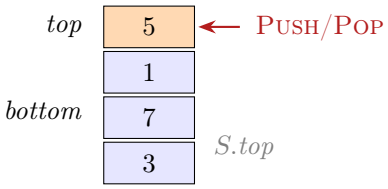
7.1 Dynamic Sets

Data structures are used to implement **dynamic sets**. Operations are grouped into two categories:

- **Modification**: Insert, Delete, ...
- **Query**: Search, Minimum, Maximum, ...

7.2 Stacks – LIFO

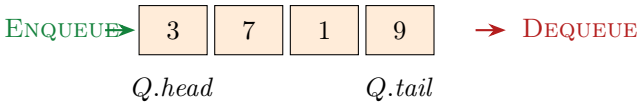
A stack follows the **LIFO** (*Last-In, First-Out*) principle: last in, first out, like a stack of plates.



- PUSH(S, x): pushes x onto the top. $O(1)$.
- POP(S): removes and returns the top element. $O(1)$.
- STACK-EMPTY(S): tests if $S.top = 0$. $O(1)$.

7.3 Queues – FIFO

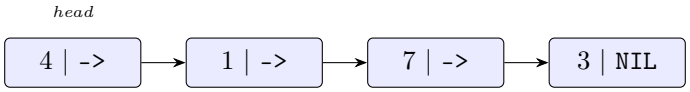
A queue follows the **FIFO** (*First-In, First-Out*) principle: first in, first out, like a waiting line.



- Enqueue(Q, x): adds x to the tail. $O(1)$.
- Dequeue(Q): removes and returns the head element. $O(1)$.

7.4 Linked Lists

Linear order is determined by **pointers** stored in each object (and not by array indices).



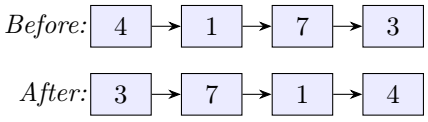
Operation	Time	Remark
List-Search(L, k)	$O(n)$	Sequential traversal
List-Insert(L, x)	$O(1)$	Insertion at the head
List-Delete(L, x)	$O(1)$	If pointer already known (doubly linked list)

Sentinels: a dummy object `L.nil` represents both ends of the list simultaneously, simplifying boundary cases (no `NULL` testing).

7.4.1 Example: reversing a list in $\Theta(n)$ time, $\Theta(1)$ space

Algorithm 3 Reverse-List(L)

```
1: prev ← NIL
2: curr ← L.head
3: while curr ≠ NIL do
4:     next ← curr.next
5:     curr.next ← prev                    // reverse the link
6:     prev ← curr
7:     curr ← next
8: end while
9: L.head ← prev
```



7.5 Conclusion on Elementary Structures

Strengths and Limitations

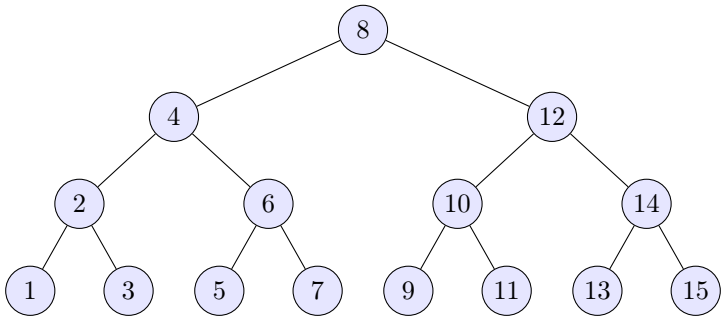
Stacks, queues, and linked lists excel at $O(1)$ insertions/deletions, but are **poor for searching** ($O(n)$). For fast searching, more sophisticated structures are needed (trees, hash tables).

8 Lecture 8 : binary tree of research & dynamic programming

8.1 Motivation: The Guessing Game

Ola is thinking of an integer between 1 and 15. For each guess, he answers *correct*, *smaller*, or *larger*.

Optimal strategy: always guess the middle of the remaining interval. This guarantees finding any number in at most $\lceil \log_2 15 \rceil = 4$ questions. This strategy is naturally encoded as a tree:



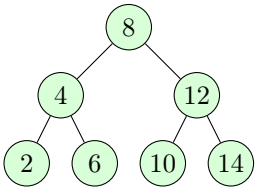
8.2 Definition and Fundamental Property

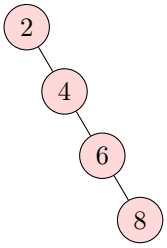
Definition 8.1: Binary Search Tree (BST)

A **BST** is a binary tree T where each node x satisfies:

- If y is in the *left subtree* of x: $y.key < x.key$.
- If y is in the *right subtree* of x: $y.key \geq x.key$.

The **height** h is the number of edges on the longest root–leaf path.





Balanced: $h = 2$, ops in $O(\log n)$

Degenerate: $h = n - 1$, ops in $O(n)$

Complexity 8.1: All operations on a BST

Search, insertion, deletion, min, max, successor, predecessor: all in $O(h)$. For a balanced tree: $h = O(\log n)$.

8.3 Queries on a BST

8.3.1 Search

Algorithm 4 BST-Search(T, k)

```
1: x ← T.root
2: while x ≠ NIL and k ≠ x.key do
3:     if k < x.key then
4:         x ← x.left
5:     else
6:         x ← x.right
7:     end if
8: end while
9: return x
```

8.3.2 Minimum, Maximum, Successor

Theorem 8.1: Locating min/max

- The **minimum** is the leftmost node.
- The **maximum** is the rightmost node.

The **successor** of x (smallest key $> x.key$) is found:

1. If x has a right subtree: it is the *minimum* of the right subtree.
2. Otherwise: go up to the first ancestor y such that x is in the *left* subtree of y.

8.4 BST Traversal

Traversal	Order	Result (ex.)	Utility
Inorder	left, root, right	$1, 2, \ldots, n$ (sorted!)	Sorting
Preorder	root, left, right	root first	Copying
Postorder	left, right, root	root last	Deletion

Key point 8.1: Inorder Traversal and Sorting

The inorder traversal of a BST always produces keys in **increasing order** (a direct property of the BST definition). Complexity: $\Theta(n)$.

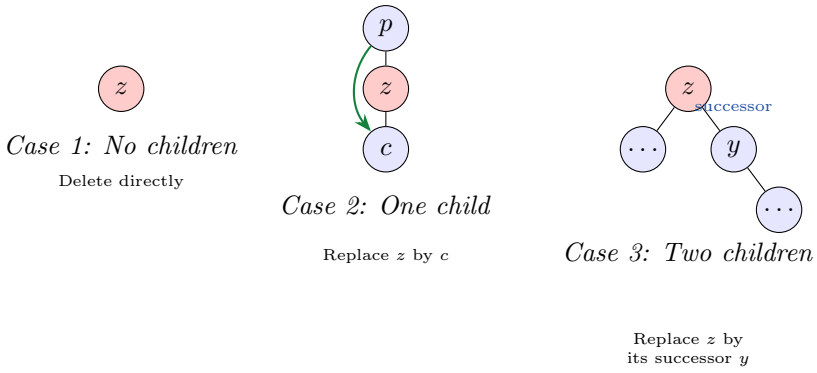
8.5 BST Modifications

8.5.1 Insertion

Search for $z.key$ until reaching NIL, then insert z at that position. Complexity: $O(h)$.

8.5.2 Deletion

Three cases according to the number of children of the node z to be deleted:



8.6 Summary of BST Operations

Operation	Time	Key Idea
Search	$O(h)$	Follow left/right according to the key
Minimum / Maximum	$O(h)$	Leftmost / rightmost node
Successor / Pred.	$O(h)$	Min of right subtree, or first ancestor
Insertion	$O(h)$	Search phase + leaf insertion
Deletion	$O(h)$	3 cases; uses TRANSPLANT
Inorder traversal	$\Theta(n)$	Visits each node once

Key point 8.2: Balancing

In the worst case $h = \Theta(n)$ (degenerate tree). Self-balancing trees (AVL, Red-Black) maintain $h = O(\log n)$, guaranteeing $O(\log n)$ per operation.

8.7 Comparison of Data Structures

Structure	Capacity	Insertion	Deletion	Search
Stack / Queue (array)	fixed	$O(1)$	$O(1)$	$O(n)$
Linked list	dynamic	$O(1)$	$O(1)$	$O(n)$
Balanced BST	dynamic	$O(\log n)$	$O(\log n)$	$O(\log n)$

Dynamic Programming

8.8 Paradigm and Central Idea

Definition 8.2: Dynamic Programming (DP)

Dynamic programming is an algorithmic paradigm (not a language feature). Its principle:

Store already performed calculations to avoid repeating them.

This allows transforming exponential algorithms into polynomial algorithms.

A DP algorithm relies on two properties:

1. **Optimal substructure**: the optimal solution to a problem contains the optimal solutions of its subproblems.
2. **Overlapping subproblems**: a naive recursive algorithm recalculates the same subproblems an exponential number of times.

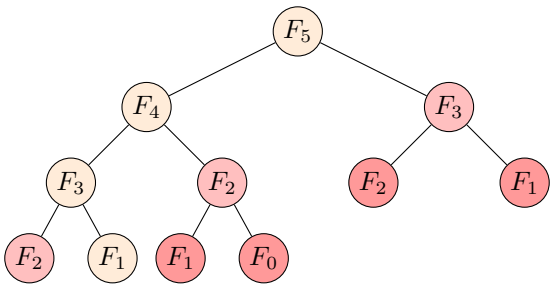
Two implementation strategies:

- **Top-down with memoization**: recursion + results table.
- **Bottom-up**: filling the table from small subproblems toward large ones.

8.9 First Example: Fibonacci Numbers

$$F_0 = 1, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \quad (n \geq 2).$$

8.9.1 Naive Recursion: Exponential Computation Tree



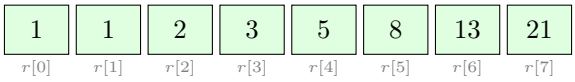
Red nodes are recalculated multiple times!

Complexity: **exponential** $O(2^n)$.

8.9.2 Bottom-up DP: $\Theta(n)$

Algorithm 5 Bottom-Up-Fib(n)

```
r[0] ← 1;  r[1] ← 1
for i = 2 to n do
    r[i] ← r[i − 1] + r[i − 2]
end for
return r[n]
```



8.10 Rod Cutting

Definition 8.3: Rod Cutting Problem

Input: a rod of length n and a table of prices p_i for a rod of length i $(1 \leq i \leq n)$.
Objective: find the cutting that maximizes total revenue.

There are 2^{n-1} possible cuttings: brute force is unfeasible.

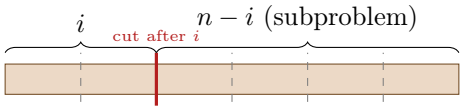
8.10.1 Optimal Substructure and Recurrence

Theorem 8.2: Optimal Substructure

If the first optimal cut is after i units, and if $s_1, \ldots, s_k$ is an optimal cutting of the remaining rod of length $n - i$, then $i, s_1, \ldots, s_k$ is optimal for the rod of length n.

Let $r(n)$ be the optimal revenue for a rod of length n:

$$r(n) = \begin{cases} 0 & \text{if } n = 0, \\ \max\limits_{1 \le i \le n} \{p_i + r(n-i)\} & \text{if } n \ge 1. \end{cases}$$



Complexity 8.2: DP for Rod Cutting

- Top-down with memoization: $O(n^2)$ (subproblem graph).
- Bottom-up: $O(n^2)$.

8.10.2 Reconstruction of the Solution

We maintain an array $s[j]$ storing the first optimal cut for $r(j)$. Then, we trace-back: cut $s[n]$, then solve for $n - s[n]$.

i	0	1	2	3	4	5	6	7	8
$r[i]$	0	1	5	8	10	13	17	18	22
$s[i]$	0	1	2	3	2	2	6	1	2

8.11 Coin Change Problem

Definition 8.4: Coin Change

Input: n denominations $0 < w_1 < \cdots < w_n$ and a target amount W.
Output: minimum number of coins to reach exactly W.

DP Recurrence:

$$c(w) = \begin{cases} 0 & \text{if } w = 0, \\ \min\limits_{\substack{1 \le j \le n \\ w_j \le w}} \{1 + c(w - w_j)\} & \text{if } w \ge 1. \end{cases}$$

Example: $w_1 = 1, w_2 = 2, w_3 = 5$, $W = 8$. Optimal: 1 coin of each $\Rightarrow$ 3 coins in total.

Bottom-up filling from $w = 0$ to W: complexity $O(nW)$.

8.12 General Recipe for DP

Design Steps for a DP Algorithm

1. **Identify choices and optimal substructure**: a choice reduces the problem to smaller subproblems, whose optimal solutions compose the global solution.
2. **Write a recurrence** for the optimal value based on the subproblems.
3. **Avoid recalculations** via memoization (top-down) or table filling (bottom-up).
4. **(Optional) Reconstruct the solution** by storing choices in an auxiliary table and tracing back.

Problem	Complexity	Table Size
Fibonacci	$\Theta(n)$	$n+1$ entries
Rod Cutting	$O(n^2)$	$n+1$ entries
Coin Change	$O(nW)$	$W+1$ entries

9 Lecture 9 : DP – rebuild and matrice chain

9.1 Reminder: Key Elements of a DP Algorithm

Dynamic programming relies on two fundamental properties:

1. **Optimal substructure**: an optimal solution is obtained by making a *choice*, which leaves one or more subproblems to be solved, and the global optimal solution solves these subproblems optimally.
2. **Overlapping subproblems**: a naive recursive algorithm recalculates the same subproblems many times.
   - **Top-down with memoization**: solve recursively by storing each result in a table.
   - **Bottom-up**: sort the subproblems and solve the smallest ones first; thus, when a subproblem is solved, the subproblems it depends on are already computed.

9.2 Rod Cutting – Reconstruction of the Optimal Solution

In Lecture 8, the DP algorithm returned only the optimal revenue $r(n)$. It is often necessary to retrieve *how* to obtain this revenue, i.e., the lengths of the pieces to be cut.

9.2.1 Extended Algorithm with Decision Table

The idea is to maintain an auxiliary table $s[0 \dots n]$: $s[j]$ stores the length of the first piece cut (the *decision*) corresponding to the optimal revenue $r[j]$.

Algorithm 6 Extended-Bottom-Up-Cut-Rod(p, n)

```
1: Create arrays r[0...n] and s[0...n]
2: r[0] ← 0
3: for j = 1 to n do
4:     q ← −∞
5:     for i = 1 to j do
6:         if q < p[i] + r[j − i] then
7:             q ← p[i] + r[j − i]                    // best revenue
8:             s[j] ← i                               // store the optimal choice
9:         end if
10:    end for
11:    r[j] ← q
12: end for
13: return r and s
```

The complexity remains $\Theta(n^2)$. The only difference from the basic algorithm is the maintenance of $s[j]$ which records the index i achieving the maximum.

9.2.2 Example: Tables r and s for $n = 8$

Using the price table p_i: 1, 5, 8, 9, 10, 17, 17, 20, 24, 30:

i	0	1	2	3	4	5	6	7	8
$r[i]$	0	1	5	8	10	13	17	18	22
$s[i]$	0	1	2	3	2	2	6	1	2

9.2.3 Extraction of the Solution

Table s stores the choices leading to an optimal solution. We reconstruct the solution by tracing back from n:

Algorithm 7 Print-Cut-Rod-Solution(p, n)

```
1: (r, s) ← Extended-Bottom-Up-Cut-Rod(p, n)
2: while n > 0 do
3:     print s[n]                                     // size of the next piece
4:     n ← n − s[n]
5: end while
```

Trace for $n = 8$:

$n = 8$ output: **2** $n = 6$ output: **6** $n = 0$

Cuts: **2** + **6**, revenue $= 5 + 17 = 22$

9.3 Summary: Rod Cutting

Summary DP – Rod Cutting

- **Choice**: where to make the leftmost cut.
- **Optimal substructure**: the remainder must be cut optimally.
- **Recurrence**:

$$r(n) = \begin{cases} 0 & \text{si } n = 0, \\ \max\limits_{1 \le i \le n} \{p_i + r(n-i)\} & \text{si } n \ge 1. \end{cases}$$

- **Complexity**: $\Theta(n^2)$ (top-down or bottom-up).
- **Reconstruction**: auxiliary table s + trace-back in $O(n)$.

9.4 When Can Dynamic Programming Be Used?

1. **Optimal substructure.**
   - An optimal solution can be constructed by combining optimal solutions of subproblems.
   - This implies that the optimal value can be expressed by a *recursive formula*.
2. **Overlapping subproblems.**
   - A naive recursive algorithm may revisit the same subproblem many times.

Key point 9.1: Coin Change Problem

DP applies directly to this classic problem. With n denominations $0 < w_1 < \cdots < w_n$ and a target amount W, we minimize the number of coins:

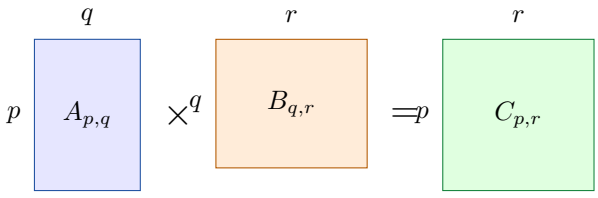
$$\min \left\{ \sum_{j=1}^{n} x_j \ : \ \sum_{j=1}^{n} w_j x_j = W, \ x_j \in \mathbb{Z}_{\ge 0} \right\}.$$

Example: $w_1 = 1$, $w_2 = 2$, $w_3 = 5$, $W = 8$. Optimal: $x_1 = x_2 = x_3 = 1 \Rightarrow 3$ coins.

Matrix Chain Multiplication

9.5 Cost of Multiplying Two Matrices

Multiplying a matrix $A_{p \times q}$ by a matrix $B_{q \times r}$ produces a matrix $C_{p \times r}$.



Cost: $p \cdot q \cdot r$ scalar multiplications

Each entry of C requires q scalar multiplications. The matrix C contains $p \cdot r$ entries, so the total cost is $\mathbf{p \cdot q \cdot r}$ **scalar multiplications**.

Why does the association order matter?

For A (20×1), B (1×10), C (10×100):
- $(A \times (B \times C))$: $1 \cdot 10 \cdot 100 + 20 \cdot 1 \cdot 100 = 3,000$
- $((A \times B) \times C)$: $20 \cdot 1 \cdot 10 + 20 \cdot 10 \cdot 100 = 20,200$

Parenthesization can **radically modify** the number of operations.

9.6 Problem Definition

Definition 9.1: Matrix Chain Multiplication

Input: a sequence $\langle A_1, A_2, \ldots, A_n \rangle$ of n matrices, where matrix A_i has dimension $p_{i-1} \times p_i$ for $i = 1, \ldots, n$.
Output: a complete parenthesization of the product $A_1 A_2 \cdots A_n$ minimizing the number of scalar multiplications.

Notes:

- We do *not* ask to compute the product, only to find the best parenthesization.
- Parenthesization can considerably affect the number of multiplications.

Example: sequence $A_1 A_2 A_3$ with dimensions 50×5, 5×100, 100×10:

Parenthesization	Cost	Calculation
$(A_1 A_2) A_3$	$75,000$	$50 \cdot 5 \cdot 100 + 50 \cdot 100 \cdot 10$
$A_1 (A_2 A_3)$	$7,500$	$5 \cdot 100 \cdot 10 + 50 \cdot 5 \cdot 10$

9.7 Optimal Substructure

Theorem 9.1: Optimal Substructure – Matrix Multiplication

Suppose the optimal parenthesization of $A_1 A_2 \cdots A_n$ performs the last multiplication as $(A_1 \cdots A_k)(A_{k+1} \cdots A_n)$. Then the parenthesizations of $A_1 \cdots A_k$ and $A_{k+1} \cdots A_n$ in this optimal solution are themselves optimal.

Proof. Let $M(P)$ be the number of scalar multiplications of a parenthesization P. Let $((O_L) \cdot (O_R))$ be the optimal solution where O_L and O_R parenthesize respectively $A_1 \cdots A_k$ and $A_{k+1} \cdots A_n$. Let P_L and P_R be optimal parenthesizations of these two subchains. We have:

$$\begin{aligned} M((O_L) \cdot (O_R)) &= p_0 \cdot p_k \cdot p_n + M(O_L) + M(O_R) \\ &\geq p_0 \cdot p_k \cdot p_n + M(P_L) + M(P_R) = M((P_L) \cdot (P_R)). \end{aligned}$$

Since $((O_L) \cdot (O_R))$ is optimal, we have equality, thus $M(P_L) = M(O_L)$ and $M(P_R) = M(O_R)$: P_L and P_R are indeed optimal. □

9.8 Recursive Formula

Let $m[i, j]$ be the minimum number of scalar multiplications to compute $A_i A_{i+1} \cdots A_j$.

$$m[i,j] = \begin{cases} 0 & \text{if } i = j, \\ \min\limits_{i \leq k < j} \{ m[i,k] + m[k+1,j] + p_{i-1}\, p_k\, p_j \} & \text{if } i < j. \end{cases}$$

Key point 9.2: Filling Order

$m[i, j]$ depends only on subproblems with a *smaller* interval $j - i$. A bottom-up algorithm therefore solves subproblems by increasing order of $j - i$ (chain length $\ell = j - i + 1$).

9.9 Bottom-up Algorithm

Algorithm 8 Matrix-Chain-Order(p)

```
1:  n ← p.length − 1
2:  Create tables m[1...n, 1...n] and s[1...n, 1...n]
3:  for i = 1 to n do
4:      m[i, i] ← 0
5:  end for
6:  for ℓ = 2 to n do                                   // ℓ = chain length
7:      for i = 1 to n − ℓ + 1 do
8:          j ← i + ℓ − 1
9:          m[i, j] ← ∞
10:         for k = i to j − 1 do
11:             q ← m[i, k] + m[k + 1, j] + p_{i−1} p_k p_j
12:             if q < m[i, j] then
13:                 m[i, j] ← q
14:                 s[i, j] ← k                         // s stores the optimal choice
15:             end if
16:         end for
17:     end for
18: end for
19: return m and s
```

Complexity 9.1: Matrix-Chain-Order

- Table m contains $O(n^2)$ entries.
- Each entry $m[i, j]$ requires $O(n)$ to find the best k.
- **Total complexity: $\Theta(n^3)$ in time, $\Theta(n^2)$ in space.**

9.10 Complete Example

Chain of $n = 6$ matrices with dimensions:

Matrix	A_1	A_2	A_3	A_4	A_5	A_6
Dimensions	30×35	35×15	15×5	5×10	10×20	20×25

Let $p = \langle 30, 35, 15, 5, 10, 20, 25 \rangle$.

Tables m (optimal costs) and s (optimal choices) computed by Matrix-Chain-Order are:

Table $m[i,j]$

$j\backslash i$	1	2	3	4	5	6
1	0					
2	15,750	0				
3	7,875	2,625	0			
4	9,375	4,375	750	0		
5	11,875	7,125	2,500	1,000	0	
6	15,125	10,500	5,375	3,500	5,000	0

Table $s[i,j]$

$j\backslash i$	1	2	3	4	5	6
2	1					
3	1	2				
4	3	3	3			
5	3	3	3	4		
6	3	3	3	5	5	

The optimal solution is $m[1,6] = 15,125$ scalar multiplications. Tracing back from $s[1,6] = 3$, we obtain the parenthesization:

$$\big(A_1\,(A_2\,A_3)\big)\big((A_4\,A_5)\,A_6\big)$$

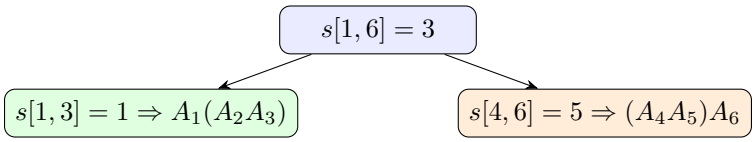
9.11 Reconstructing the Optimal Parenthesization

Table s allows for recursively reconstructing the parenthesization:

Algorithm 9 Print-Optimal-Parens(s, i, j)

```
1: if i = j then
2:     print "A_i"
3: else
4:     print "("
5:     PRINT-OPTIMAL-PARENS(s, i, s[i, j])
6:     PRINT-OPTIMAL-PARENS(s, s[i, j] + 1, j)
7:     print ")"
8: end if
```

Trace for the example ($i = 1$, $j = 6$):



Result: $\big(A_1\,(A_2\,A_3)\big)\big((A_4\,A_5)\,A_6\big)$

9.12 Summary: Matrix Chain Multiplication

Summary DP – Matrix Chain Multiplication

- **Choice**: where to place the outermost parenthesis $(A_1 \cdots A_k)(A_{k+1} \cdots A_n)$.
- **Optimal substructure**: both subchains must be parenthesized optimally (proven by the cut-and-paste argument).
- **Recurrence**:

$$m[i,j] = \begin{cases} 0 & \text{si } i = j, \\ \min\limits_{i \le k < j} \{m[i,k] + m[k+1,j] + p_{i-1}\, p_k\, p_j\} & \text{si } i < j. \end{cases}$$

- **Complexity**: $\Theta(n^3)$ in time, $\Theta(n^2)$ in space.
- **Reconstruction**: auxiliary table s + PRINT-OPTIMAL-PARENS.

9.13 General Recipe for DP (Consolidation)

Problem	Choice	Complexity	Table Size
Rod Cutting	Left cut position	$\Theta(n^2)$	$n+1$ entries
Coin Change	Denomination used	$O(nW)$	$W+1$ entries
Matrix Chains	Parenthesis position	$\Theta(n^3)$	n^2 entries

The three systematic steps remain:

1. Identify choices and optimal substructure.
2. Write the recurrence for the optimal value.
3. Solve efficiently (top-down with memoization or bottom-up).

10 Lecture 10 : DP – longest common subsequence

Longest Common Subsequence (LCS)

10.1 Definition and Motivation

Definition 10.1: Longest Common Subsequence

Input: two sequences $X = \langle x_1, \ldots, x_m \rangle$ and $Y = \langle y_1, \ldots, y_n \rangle$.
Output: a common subsequence to both whose length is maximal.
Reminder: a subsequence does not need to be consecutive, but must respect the order of the elements.

Example: for the words `heroically` and `scholarly`, the LCS is `holly` (length 5): the letters appear in the correct order in each of the two words, without necessarily being adjacent.

10.2 Why Naive Approaches Fail

Brute force: for each subsequence of X, check if it is a subsequence of Y.

Complexity 10.1: Brute force for LCS

- X has 2^m subsequences.
- Each verification takes $\Theta(n)$ (traversing Y letter by letter).
- **Total complexity:** $\Theta(n \cdot 2^m)$ — **exponential, unusable.**

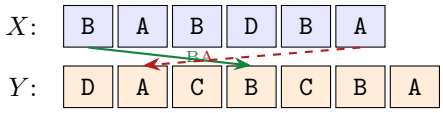
There is also no natural greedy algorithm for this problem: the locally best letter does not necessarily lead to the global optimal solution. Dynamic programming is the right tool.

10.3 Intuition of the DP Approach

We traverse the two sequences from the *right* (the end) to the left, step by step.

At each position (i, j) (looking at x_i and y_j):

- **If** $x_i = y_j$: this letter is part of the LCS; we include it and solve the subproblem on prefixes X_{i-1} and Y_{j-1}.
- **If** $x_i \neq y_j$: the optimal LCS is obtained by moving one step to the left in *one* of the two sequences, and taking the best of the two results.



10.4 Optimal Substructure

Let $X_i = \langle x_1, x_2, \ldots, x_i \rangle$ and $Y_j = \langle y_1, y_2, \ldots, y_j \rangle$ be the prefixes of length i and j.

Theorem 10.1: Optimal Substructure for LCS

Let $Z = \langle z_1, z_2, \ldots, z_k \rangle$ be an LCS of X_i and Y_j.

1. **If** $x_i = y_j$: then $z_k = x_i = y_j$ and Z_{k-1} is an LCS of X_{i-1} and Y_{j-1}.
2. **If** $x_i \neq y_j$ **and** $z_k \neq x_i$: then Z is an LCS of X_{i-1} and Y_j.
3. **If** $x_i \neq y_j$ **and** $z_k \neq y_j$: then Z is an LCS of X_i and Y_{j-1}.

Proof. **Case 1 ($x_i = y_j$).** If $z_k \neq x_i = y_j$, then $Z' = \langle z_1, \ldots, z_k, x_i \rangle$ is a common subsequence of X_i and Y_j of length $k+1$, which contradicts the optimality of Z. Thus $z_k = x_i = y_j$.

Now suppose that Z_{k-1} is not an LCS of X_{i-1} and Y_{j-1}: there would exist a common subsequence W of X_{i-1} and Y_{j-1} of length $\geq k$. Then $\langle W, z_k \rangle$ would be a common subsequence of X_i and Y_j of length $\geq k+1$, a contradiction.

Case 2 ($x_i \neq y_j$, $z_k \neq x_i$). Z is already a common subsequence of X_{i-1} and Y_j. If it were not optimal, there would exist W of length $> k$ as a subsequence of X_{i-1} and Y_j, which would also be a subsequence of X_i and Y_j (since W does not end with x_i), contradicting the optimality of Z. Case 3 is symmetric. □

10.5 Recursive Formulation

It follows from the previous theorem that the length of the LCS of X_i and Y_j is:

$$c[i, j] = \text{length of the LCS of } X_i \text{ and } Y_j$$

with the recurrence:

$$c[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ c[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j, \\ \max\big(c[i-1, j],\ c[i, j-1]\big) & \text{if } i, j > 0 \text{ and } x_i \neq y_j. \end{cases}$$

We want to compute $c[m, n]$.

Key point 10.1: Overlapping Subproblems

A naive recursion would solve the same subproblems an exponential number of times. The bottom-up approach fills the table c only once, by processing the subproblems by increasing size (rows i from 0 to m, columns j from 0 to n).

10.6 Bottom-up Algorithm and Complete Example

For $X = \langle B, A, B, D, B, A \rangle$ and $Y = \langle D, A, C, B, C, B, A \rangle$ ($m = 6$, $n = 7$), here is the complete table $c[i, j]$:

		j							
i		**0**	**1 (D)**	**2 (A)**	**3 (C)**	**4 (B)**	**5 (C)**	**6 (B)**	**7 (A)**
0		0	0	0	0	0	0	0	0
1 (B)		0	0	0	0	1	1	1	1
2 (A)		0	0	1	1	1	1	1	2
3 (B)		0	0	1	1	2	2	2	2
4 (D)		0	1	1	1	2	2	2	2
5 (B)		0	1	1	1	2	2	3	3
6 (A)		0	1	2	2	2	2	3	**4**

The LCS has length **4** (value $c[6, 7]$). By tracing back the decisions, we obtain the LCS **ABBA**.

Key point 10.2: Reading the Table

To fill $c[i, j]$: if $x_i = y_j$, come from the diagonal ($c[i-1, j-1] + 1$). Otherwise, take the maximum of the cell above ($c[i-1, j]$) and the cell to the left ($c[i, j-1]$).

10.7 Pseudocode

Algorithm 10 LCS-Length(X, Y)

```
1:  m ← X.length;  n ← Y.length
2:  Create tables c[0...m, 0...n] and b[1...m, 1...n]
3:  for i = 0 to m do
4:      c[i, 0] ← 0
5:  end for
6:  for j = 0 to n do
7:      c[0, j] ← 0
8:  end for
9:  for i = 1 to m do
10:     for j = 1 to n do
11:         if x_i = y_j then
12:             c[i, j] ← c[i − 1, j − 1] + 1
13:             b[i, j] ← " ↖ "                    // diagonal: common letter
14:         else if c[i − 1, j] ≥ c[i, j − 1] then
15:             c[i, j] ← c[i − 1, j]
16:             b[i, j] ← " ↑ "                    // up: advance in X
17:         else
18:             c[i, j] ← c[i, j − 1]
19:             b[i, j] ← " ← "                    // left: advance in Y
20:         end if
21:     end for
22: end for
23: return c and b
```

Complexity 10.2: LCS-Length

- Table c contains $(m+1)(n+1)$ entries, each calculated in $\Theta(1)$.
- **Time complexity:** $\Theta(m \cdot n)$.
- **Space complexity:** $\Theta(m \cdot n)$.

10.8 LCS Reconstruction

Table b stores the optimal choices: we trace it back recursively from $b[m, n]$ toward the upper-left corner.

Algorithm 11 Print-LCS(b, X, i, j)

```
1:  if i = 0 or j = 0 then
2:      return
3:  end if
4:  if b[i, j] = " ↖ " then
5:      PRINT-LCS(b, X, i − 1, j − 1)
6:      print x_i                                  // common letter: we include it
7:  else if b[i, j] = " ↑ " then
8:      PRINT-LCS(b, X, i − 1, j)
9:  else
10:     PRINT-LCS(b, X, i, j − 1)
11: end if
```

Complexity 10.3: Print-LCS

- Each recursive call decreases $i + j$ by at least 1.
- The recurrence satisfies $T(n) \leq T(n-1) + \Theta(1)$, hence $T(n) = O(n)$.
- **Total complexity:** $\Theta(|X| + |Y|)$ (the lower bound comes from $T(n) \geq T(n-2) + \Theta(1)$).

10.9 Backtracking Illustration

For $X = \langle B, A, B, D, B, A \rangle$, $Y = \langle D, A, C, B, C, B, A \rangle$, the path in table b (illustrated below) identifies the letters **A, B, B, A** = **ABBA** as the LCS:

		D	A	C	B	C	B	A
	0	0	0	0	0	0	0	0
B	0	0	0	0	1	1	1	1
A	0	0	1	1	1	1	1	2
B	0	0	1	1	2	2	2	2
D	0	1	1	1	2	2	2	2
B	0	1	1	1	2	2	3	3
A	0	1	2	2	2	2	3	4

LCS = ABBA (length 4)

10.10 Summary: LCS

Summary DP – Longest Common Subsequence

- **Choice**: if $x_i = y_j$, include this letter; otherwise advance in X or in Y (take the best of the two).
- **Optimal substructure**: 3 cases depending on whether $x_i = y_j$ or not (theorem above).
- **Recurrence**:

$$c[i,j] = \begin{cases} 0 & i = 0 \text{ or } j = 0, \\ c[i-1, j-1] + 1 & x_i = y_j, \\ \max(c[i-1, j],\, c[i, j-1]) & x_i \neq y_j. \end{cases}$$

- **Complexity**: $\Theta(mn)$ in time and space.
- **Reconstruction**: table b + PRINT-LCS in $\Theta(|X| + |Y|)$.

10.11 General Overview of Dynamic Programming

DP Recipe in Three Steps

1. **Identify the choices and the optimal substructure.**
2. **Write the optimal value as a recurrence on subproblems.**
3. **Solve efficiently**: top-down with memoization or bottom-up (without recalculating the same subproblems).

Problem	Choice	Complexity	Table
Rod Cutting	Left cut position	$\Theta(n^2)$	$n+1$
Coin Change	Denomination used	$O(nW)$	$W+1$
Matrix Chains	Parenthesis position	$\Theta(n^3)$	n^2
LCS	Include $x_i = y_j$ / advance	$\Theta(mn)$	$(m+1)(n+1)$

11 Lecture 11 : DP – binary tree of optimal research

Optimal Binary Search Trees (OBST)

11.1 Motivation

In a search engine, certain terms are searched much more often than others. If we store keys in a BST, the search order depends on the tree's structure. Placing the most frequent keys near the root minimizes the average cost of a search. The OBST problem formalizes this idea.

11.2 Problem Definition

Definition 11.1: Optimal Binary Search Tree

Input: a sorted sequence of n distinct keys $K = \langle k_1 < k_2 < \cdots < k_n \rangle$, with a search probability $p_i > 0$ for each k_i (where $\sum_{i=1}^{n} p_i = 1$).
Objective: construct a BST T on these keys that minimizes the expected search cost:

$$E[\text{search cost in } T] = \sum_{i=1}^{n} (\text{depth}_T(k_i) + 1)\, p_i = 1 + \sum_{i=1}^{n} \text{depth}_T(k_i) \cdot p_i.$$

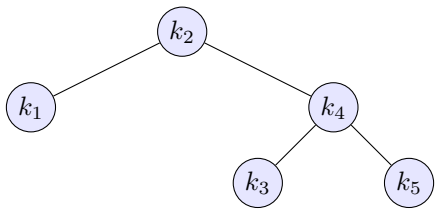
The actual cost of searching for k_i is $\text{depth}_T(k_i) + 1$ (the number of nodes examined, including the root).

11.3 Motivating Example

With $n = 5$ keys and the probabilities:

i	1	2	3	4	5
p_i	0.25	0.2	0.05	0.2	0.3

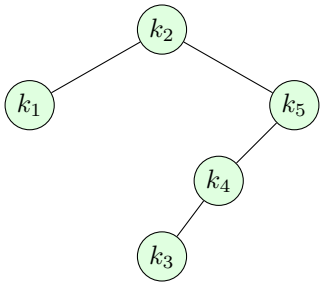
Tree A (root k_2):



i	$\text{depth}_T(k_i)$	$\text{depth}_T(k_i) \cdot p_i$
1	1	0.25
2	0	0
3	2	0.10
4	1	0.20
5	2	0.60
		$\sum = 1.15$

$E[\text{cost}] = 1 + 1.15 = 2.15$.

Tree B (root k_2, but k_5 right child of k_2, k_4 left of k_5, k_3 left of k_4):



i	$\text{depth}_T(k_i)$	$\text{depth}_T(k_i) \cdot p_i$
1	1	0.25
2	0	0
3	3	0.15
4	2	0.40
5	1	0.30
		$\sum = 1.10$

$E[\text{cost}] = 1 + 1.10 = \mathbf{2.10}$ — this tree turns out to be optimal.

Key point 11.1: Key Observations

- The OBST is *not* necessarily the tree with minimum height.
- The key with the highest probability is *not* necessarily at the root.
- The number of BSTs with n nodes is exponential (Catalan numbers $\sim 4^n/n^{3/2}$), so brute force is impractical.

11.4 Optimal Substructure

A BST is built by choosing a root k_r, then recursively building the left subtree on $k_i, \ldots, k_{r-1}$ and the right subtree on $k_{r+1}, \ldots, k_j$.

Theorem 11.1: Optimal Substructure of OBST

If we build an optimal BST on $k_i, \ldots, k_j$ with root k_r, then the left and right subtrees must also be OBSTs (on $k_i, \ldots, k_{r-1}$ and $k_{r+1}, \ldots, k_j$ respectively).

Cost analysis. Letting $w(i, j) = \sum_{\ell=i}^{j} p_\ell$ be the sum of probabilities over $k_i, \ldots, k_j$, we observe that when a subtree becomes the left or right child of the root, the depth of each of its nodes

increases by 1. Therefore:

$$E[\text{cost of subtree } k_i \dots k_j] = p_r + \big(w(i, r-1) + E[\text{cost of left}]\big) + \big(w(r+1, j) + E[\text{cost of right}]\big).$$

By regrouping, we get:

$$E[\text{cost}] = p_r + w(i, r-1) + E[\text{cost of left}] + w(r+1, j) + E[\text{cost of right}] = w(i, j) + E[\text{cost of left}] + E[\text{cost of right}].$$

11.5 Recursive Formulation

Let us define $e[i, j]$ as the expected search cost of the OBST built on $k_i, \dots, k_j$, and $w(i, j) = \sum_{\ell=i}^{j} p_\ell$.

$$e[i, j] = \begin{cases} 0 & \text{if } i = j + 1 \quad \text{(empty subtree)}, \\ \min\limits_{i \le r \le j} \big\{ e[i, r-1] + e[r+1, j] + w(i, j) \big\} & \text{if } i \le j. \end{cases}$$

We want to calculate $e[1, n]$.

Key point 11.2: Link with the Sum of Probabilities

The term $w(i, j) = \sum_{\ell=i}^{j} p_\ell$ can be precalculated incrementally: $w(i, j) = w(i, j-1) + p_j$, which avoids recalculating it at each step.

11.6 Bottom-up Algorithm

We fill tables e and r (optimal roots) by increasing chain length ($\ell = j - i + 1$ from 1 to n):

Algorithm 12 Optimal-BST(p, n)

```
Create tables e[1...n+1, 0...n], w[1...n+1, 0...n] and r[1...n, 1...n]
for i = 1 to n + 1 do
    e[i, i − 1] ← 0                                   // empty subtrees
    w[i, i − 1] ← 0
end for
for ℓ = 1 to n do                                     // chain length
    for i = 1 to n − ℓ + 1 do
        j ← i + ℓ − 1
        e[i, j] ← ∞
        w[i, j] ← w[i, j − 1] + p_j
        for r = i to j do                             // try each root
            t ← e[i, r − 1] + e[r + 1, j] + w[i, j]
            if t < e[i, j] then
                e[i, j] ← t
                root[i, j] ← r                        // store optimal root
            end if
        end for
    end for
end for
return e and root
```

Complexity 11.1: Optimal-BST

- Table e contains $O(n^2)$ entries.
- Each entry $e[i, j]$ requires $O(n)$ to test all roots r.
- **Total complexity: $\Theta(n^3)$ in time, $\Theta(n^2)$ in space.**

11.7 Complete Numerical Example

With $p = (0.25, 0.2, 0.05, 0.2, 0.3)$, the table $e[i, j]$ filled row by row (indices j in columns, i in rows):

$e[i,j]$	$j=0$	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$
$i=1$	0	0.25	0.65	0.80	1.25	**2.10**
$i=2$		0	0.20	0.30	0.75	1.35
$i=3$			0	0.05	0.30	0.85
$i=4$				0	0.20	0.70
$i=5$					0	0.30
$i=6$						0

The OBST has an expected search cost of $e[1, 5] = \mathbf{2.10}$.

Calculation of $e[1, 2]$ for illustration: $w(1, 2) = p_1 + p_2 = 0.45$. We test:

- $r = 1$: $e[1, 0] + e[2, 2] + 0.45 = 0 + 0.20 + 0.45 = 0.65$
- $r = 2$: $e[1, 1] + e[3, 2] + 0.45 = 0.25 + 0 + 0.45 = 0.70$

Minimum: $e[1, 2] = 0.65$ with $r = 1$ (root k_1).

11.8 Reconstructing the Optimal Tree

Table $root[i, j]$ stores the chosen root at each step. We reconstruct the tree recursively:

Algorithm 13 Build-Optimal-BST(*root*, i, j, *parent*)

```
1: if i > j then
2:     return                                          // empty subtree
3: end if
4: r ← root[i, j]
5: print "k_r is {root / left child / right child} of parent"
6: BUILD-OPTIMAL-BST(root, i, r − 1, k_r)              // left subtree
7: BUILD-OPTIMAL-BST(root, r + 1, j, k_r)              // right subtree
```

11.9 Summary: OBST

Summary DP – Optimal Binary Search Tree

- **Choice**: which key k_r to place at the root of the subtree $k_i, \ldots, k_j$.
- **Optimal substructure**: the left and right subtrees must themselves be OBSTs.
- **Recurrence**:

$$e[i,j] = \begin{cases} 0 & i = j+1, \\ \min\limits_{i \le r \le j} \{e[i,r-1] + e[r+1,j] + w(i,j)\} & i \le j. \end{cases}$$

- **Complexity**: $\Theta(n^3)$ in time, $\Theta(n^2)$ in space.
- **Reconstruction**: table $root[i,j]$ + recursive calls.

11.10 General Summary Table for DP

Problem	Choice	Complexity	Table
Rod Cutting	Left cut position	$\Theta(n^2)$	$n+1$
Coin Change	Denomination used	$O(nW)$	$W+1$
Matrix Chains	Parenthesis position	$\Theta(n^3)$	n^2
LCS	Include $x_i = y_j$ / advance	$\Theta(mn)$	$(m+1)(n+1)$
OBST	Root of subtree $k_i \ldots k_j$	$\Theta(n^3)$	n^2

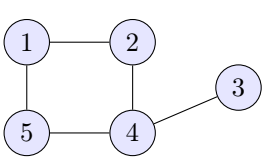
12 Lecture 12 : Graphs – BFS and DFS

Graphs

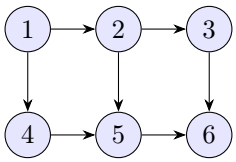
12.1 Definitions and Representations

Definition 12.1: Graph

A **graph** $G = (V, E)$ consists of a set of **vertices** V and a set of **edges** E containing pairs (ordered or unordered) of vertices. A graph can be *undirected*, *directed*, *vertex-weighted*, *edge-weighted*, etc.



Undirected graph



Directed graph

12.1.1 Adjacency Lists

We maintain an array Adj of $|V|$ lists: the list for u contains all vertices v such that $(u, v) \in E$. We denote this as $G.Adj[u]$ in algorithms.

Operation	Cost	
Space	$\Theta(V+E)$	compact for sparse graphs
List neighbors of u	$\Theta(\deg(u))$	
Test $(u,v) \in E$	$O(\deg(u))$	

12.1.2 Adjacency Matrix

A $|V| \times |V|$ matrix where $a_{ij} = 1$ if $(i,j) \in E$, and 0 otherwise.

Operation	Cost	
Space	$\Theta(V^2)$	dense, simple to implement
List neighbors of u	$\Theta(V)$	
Test $(u,v) \in E$	$\Theta(1)$	

Key point 12.1: Choice of Representation

Both representations can be extended to weighted graphs. For *sparse* graphs ($|E| \ll |V|^2$), adjacency lists are preferable for space. The matrix is advantageous when edge membership is tested frequently.

12.2 Breadth-First Search (BFS)

Definition 12.2: Breadth-First Search (BFS)

Input: graph $G = (V, E)$ (directed or undirected), source vertex $s \in V$.
Output: for every vertex v, $v.d$ = shortest distance (in number of edges) from s to v.

Idea: send out a wave from s. It reaches all vertices at distance 1 first, then at distance 2, etc. We use a *queue* (FIFO).

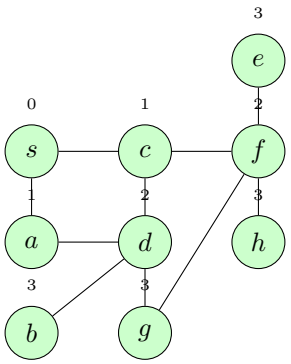
12.2.1 Pseudocode

Algorithm 14 BFS(G, s)

```
1:  for each vertex u ∈ G.V \ {s} do
2:      u.d ← ∞
3:  end for
4:  s.d ← 0
5:  Q ← ∅;  ENQUEUE(Q, s)
6:  while Q ≠ ∅ do
7:      u ← DEQUEUE(Q)
8:      for each v ∈ G.Adj[u] do
9:          if v.d = ∞ then                    // v not yet discovered
10:             v.d ← u.d + 1
11:             ENQUEUE(Q, v)
12:         end if
13:     end for
14: end while
```

12.2.2 BFS Example

Graph with 9 vertices $\{s, a, b, c, d, e, f, g, h\}$ from source s:



The discovery order is: $s \to a, c \to d \to f, b \to e, g, h$. The final distances are indeed the distances in the graph.

12.2.3 Analysis

Complexity 12.1: BFS

- Each vertex is enqueued at most once: $O(V)$.
- Each edge is examined at most once (directed) or twice (undirected): $O(E)$.
- **Total complexity:** $O(V + E)$**.**

Note: BFS does not necessarily visit all vertices (only those reachable from s). One can record the *BFS tree* by storing the edge that discovered each vertex ($v.\pi \leftarrow u$).

12.3 Depth-First Search (DFS)

Definition 12.3: Depth-First Search (DFS)

Input: graph $G = (V, E)$, directed or undirected.
Output: for every vertex v, two timestamps: $v.d$ (*discovery* time) and $v.f$ (*finishing* time).

Idea: explore each edge as far as possible before backtracking. As soon as a vertex is discovered, we explore from it (unlike BFS which explores nearby vertices first). We restart from a new undiscovered vertex if necessary.

12.3.1 Vertex Colors

During execution, each vertex has a color:

- **WHITE**: undiscovered.
- **GRAY**: discovered but not finished (exploration in progress from it).
- **BLACK**: finished (everything reachable from it has been explored).

12.3.2 Pseudocode

Algorithm 15 DFS(G)

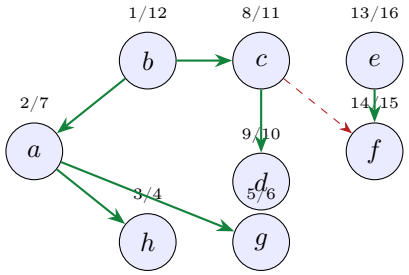
```
1: for each vertex u ∈ G.V do
2:     u.color ← WHITE
3: end for
4: time ← 0
5: for each vertex u ∈ G.V do
6:     if u.color = WHITE then
7:         DFS-Visit(G, u)
8:     end if
9: end for
```

Algorithm 16 DFS-Visit(G, u)

```
1: time ← time + 1
2: u.d ← time                                   // discovery timestamp
3: u.color ← GRAY
4: for each v ∈ G.Adj[u] do
5:     if v.color = WHITE then
6:         DFS-Visit(G, v)
7:     end if
8: end for
9: u.color ← BLACK
10: time ← time + 1
11: u.f ← time                                  // finishing timestamp
```

12.3.3 DFS Example

Graph with 8 vertices $\{a, b, c, d, e, f, g, h\}$, timestamps d/f:



The DFS forest here contains two trees rooted at b (time 1) and e (time 13).

12.3.4 Analysis

Complexity 12.2: DFS

- Each vertex is discovered exactly once: $\Theta(V)$.
- Each edge is examined exactly once (directed) or twice (undirected): $\Theta(E)$.
- **Total complexity:** $\Theta(V + E)$**.**

DFS produces a **DFS forest**: one or more trees, each originating from a DFS-Visit call in the main loop.

12.4 Edge Classification (DFS)

In a DFS, each edge (u, v) can be classified according to its timestamps:

Type	Definition	Condition
Tree edge	in the DFS forest	v was WHITE when (u, v) explored
Back edge	(u, v): u is descendant of v	$v.d < u.d < u.f < v.f$
Forward edge	(u, v): v is descendant of u,but not via tree	$u.d < v.d < v.f < u.f$
Cross edge	any other edge	$v.f < u.d$

Key point 12.2: Undirected Graphs

In the DFS of an *undirected* graph, we only obtain tree edges and back edges — never forward edges or cross edges. Indeed, if (u, v) is examined from u and v is already GRAY or BLACK, then v is necessarily an ancestor of u in the current tree, which results in a back edge.

12.5 Summary: BFS and DFS

	BFS	**DFS**
Structure used	Queue (FIFO)	Stack (recursion)
Exploration order	By increasing distance	Depth-first
Main output	Distances $v.d$ from s	Timestamps $v.d$, $v.f$
Complexity	$O(V + E)$	$\Theta(V + E)$
Reaches whole graph	No (from s only)	Yes (if looping through all vertices)
Applications	Shortest path (unweighted)	Topological sort, connected components

13 Lecture 13 : Graphs – DFS theorem and topological sort

13.1 Parenthesis Theorem

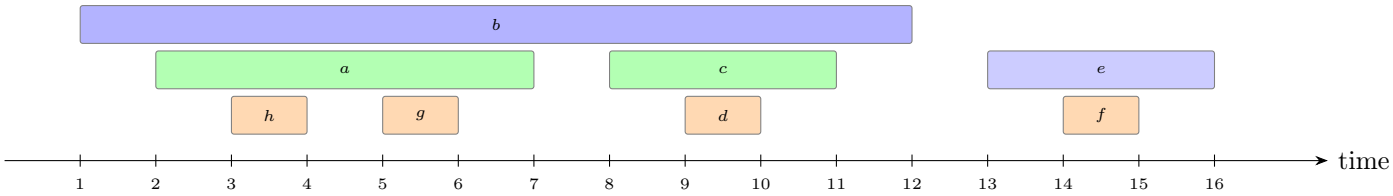
DFS timestamps have a strict nested structure:

Theorem 13.1: Parenthesis Theorem

For any two vertices u and v, exactly one of the following three situations occurs:

1. The intervals $[u.d, u.f]$ and $[v.d, v.f]$ are **disjoint**, and neither u nor v is an ancestor of the other.
2. $[v.d, v.f] \subset [u.d, u.f]$ (i.e., $u.d < v.d < v.f < u.f$) and v is a **descendant** of u.
3. $[u.d, u.f] \subset [v.d, v.f]$ (i.e., $v.d < u.d < u.f < v.f$) and u is a **descendant** of v.

Illustration. Using the example from Lecture 12 (b: 1/12, a: 2/7, h: 3/4, g: 5/6, c: 8/11, d: 9/10, e: 13/16, f: 14/15):



We see that the intervals are either nested (ancestor/descendant) or disjoint (no descendant relationship). They never partially overlap.

13.2 White-Path Theorem

Theorem 13.2: White-Path Theorem

In a DFS forest of a graph G, vertex v is a **descendant** of u if and only if at time $u.d$ (when u is just discovered), there exists a path from u to v consisting **entirely of WHITE vertices** (not yet discovered), with the exception of u itself which has just been colored GRAY.

This theorem fully characterizes the descendant relationship in the DFS forest: v will be in u's subtree precisely if such a white path exists at the moment u is discovered.

Topological Sort

13.3 Definition

Definition 13.1: Topological Sort

Input: a directed acyclic graph (DAG) $G = (V, E)$.
Output: a linear ordering of vertices such that for every edge $(u, v) \in E$, u appears before v in the ordering.

Motivating example: the order in which one gets dressed in the morning. If you must put on socks before shoes, and underwear before pants, a topological sort provides an order consistent with all these constraints.

Key point 13.1: DAG and Applicability

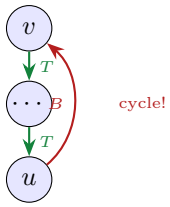
A topological sort exists only if the graph is **acyclic** (DAG). In the presence of a cycle, no linear ordering can satisfy all precedence constraints.

13.4 Characterizing DAGs via DFS

Theorem 13.3: DAG $\Leftrightarrow$ No DFS Back Edge

A directed graph G is acyclic if and only if a DFS of G produces **no back edges**.

Proof. **Back edge $\Rightarrow$ cycle.** Suppose a back edge (u, v) exists. Then v is an ancestor of u in the DFS forest, so there is a path from v to u in G. The back edge (u, v) closes this path into a cycle.



Cycle ⇒ back edge. Let C be a cycle in G and v be the first vertex of C discovered by DFS. Let (u, v) be the edge preceding v in the cycle. At time $v.d$, the vertices of C form a white path from v to u. By the white-path theorem, u becomes a descendant of v, so the edge (u, v) is a back edge. □

13.5 Algorithm

Algorithm 17 Topological-Sort(G)

1: Call DFS(G) to compute finishing times $v.f$ for all v
2: **return** the vertices in **decreasing** order of finishing times $v.f$

Efficient implementation: explicit sorting is not necessary. One can insert each vertex at the *head* of a linked list at the moment it is finished (colored BLACK). At the end of the DFS, the list contains the vertices in topological order.

Complexity 13.1: Topological-Sort

Complexity: $\Theta(V + E)$ — identical to DFS complexity.

13.6 Proof of Correctness

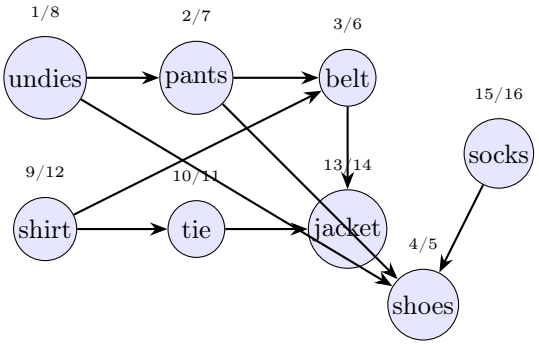
We must show that if $(u, v) \in E$, then $v.f < u.f$ (so u appears before v in the decreasing order of finishing times).

When edge (u, v) is explored during DFS, u is GRAY. Three cases for v:

Color of v	**Consequence**	**Result**
GRAY	v would be an ancestor of u	Back edge ⇒ cycle ⇒ contradiction
WHITE	v becomes descendant of u	By parenthesis: $u.d < v.d < v.f < u.f$
BLACK	v already finished, u not yet	Directly $v.f < u.f$

In all non-contradictory cases, we indeed obtain $v.f < u.f$. □

13.7 Example



Topo. Order: $f = 16, 15, 14, 12, 11, 8, 7, 6, 5$

The topological sort (decreasing finishing times) yields a valid dressing order: each garment appears after those that must be put on before it.

13.8 Lecture Summary

Key Points

- **Parenthesis Theorem:** the intervals $[u.d, u.f]$ are either nested or disjoint — never partially overlapping.
- **White-Path Theorem:** v is a descendant of u iff there exists a white path from u to v at time $u.d$.
- **DAG $\Leftrightarrow$ No back edge:** a tool for detecting cycles.
- **Topological Sort:** DFS + decreasing order of finishing times. Complexity $\Theta(V + E)$.

14 Lecture 14 : Composantes Fortement Connexes et Réseaux de Flot

Strongly Connected Components (SCC)

14.1 Definition

Definition 14.1: Strongly Connected Component

A **strongly connected component (SCC)** of a directed graph $G = (V, E)$ is a *maximal* set of vertices $C \subseteq V$ such that for every $u, v \in C$, there is a path from u to v *and* a path from v to u (denoted $u \rightsquigarrow v$ and $v \rightsquigarrow u$).

Example: on a graph with 10 vertices $\{a, b, c, d, e, f, g, h, i, j\}$, the SCCs may group as $\{a, b, e, f\}$, $\{c, d, h\}$, $\{g\}$, $\{i, j\}$ depending on the edges present. A subset that is not maximal, or in which two vertices are not mutually reachable, is not an SCC.

14.2 Component Graph

Definition 14.2: Component Graph G^{SCC}

For a directed graph $G = (V, E)$, the component graph $G^{SCC} = (V^{SCC}, E^{SCC})$ is defined by:

- V^{SCC} contains one vertex for each SCC of G.
- E^{SCC} contains an edge between two SCCs if there exists an edge between the corresponding SCCs in G.

Theorem 14.1: G^{SCC} is a DAG

The component graph G^{SCC} is always a directed acyclic graph.

This follows from the fact that if a cycle existed in G^{SCC}, all involved SCCs would form a single SCC, which contradicts their definition.

14.3 Kosaraju's Algorithm

Algorithm 18 SCC(G)

1: Call DFS(G) to compute finishing times $u.f$ for all u
2: Compute the transpose graph G^T *// reverse all edges*
3: Call DFS(G^T) processing vertices in **decreasing** order of $u.f$ calculated in step 1
4: **return** each tree of the DFS forest from step 3 as a distinct SCC

Transpose Graph G^T: $G^T = (V, E^T)$ where $E^T = \{(u, v) : (v, u) \in E\}$ — all edges are reversed. G^T is calculated in $\Theta(V + E)$ using adjacency lists, and G and G^T possess exactly the same SCCs.

Complexity 14.1: Kosaraju's SCC

Each step executes in $\Theta(V + E)$, so the **total complexity is** $\Theta(V + E)$.

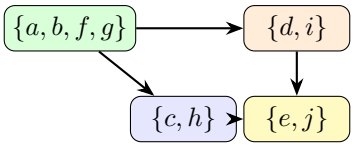
14.3.1 Intuition of Correctness

The first DFS orders the SCCs in topological order in G^{SCC} (which is a DAG). The second DFS on G^T starts from the SCC with the largest $u.f$, and in G^T the edges between SCCs are reversed: one cannot therefore "escape" to another SCC. Each tree of the second DFS is exactly one SCC.

14.3.2 Example

On a graph with 10 vertices, the first DFS (on G) produces the following timestamps (reading the slides: d: 1/12, i: 10/11, h: 2/5, c: 3/4, j: 7/8, e: 6/9, a: 14/19, b: 15/16, f: 17/18, g: 13/20).

The second DFS on G^T starts from g ($f = 20$), then a ($f = 19$), etc., and each tree formed corresponds to an SCC:



G^{SCC} (DAG)

Flow Networks

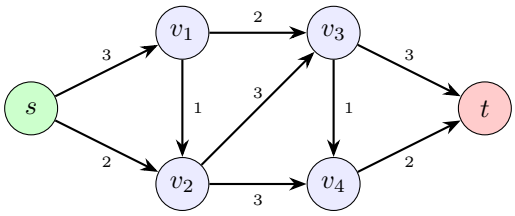
14.4 Definition

Definition 14.3: Flow network

A **flow network** is a directed graph $G = (V, E)$ where:

- Each edge (u, v) has a **capacity** $c(u, v) \geq 0$ (and $c(u, v) = 0$ if $(u, v) \notin E$).
- There is a **source** s and a **sink** t.
- We assume the absence of anti-parallel edges (without loss of generality).

Example: transporting Gruyère cheese (s) to Lausanne (t) via intermediate cities, each route having a maximum capacity.



14.4.1 Removing anti-parallel edges

If the graph contains (u, v) and (v, u), we replace (u, v) with two edges (u, v') and (v', v) with $c(u, v') = c(v', v) = c(u, v)$, by introducing a dummy vertex v'. Repeat for each anti-parallel pair ($O(E)$ times).

14.5 Definition of a Flow

Definition 14.4: Flow

A **flow** is a function $f : V \times V \to \mathbb{R}$ satisfying:
Capacity constraint: $\forall\, u, v \in V$,

$$0 \leq f(u, v) \leq c(u, v).$$

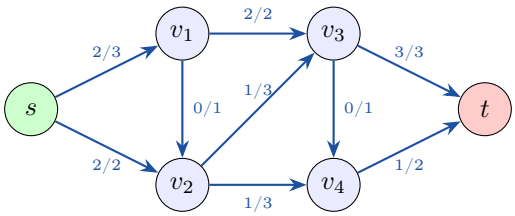
Flow conservation: $\forall\, u \in V \setminus \{s, t\}$,

$$\underbrace{\sum_{v \in V} f(v, u)}_{\text{incoming flow}} = \underbrace{\sum_{v \in V} f(u, v)}_{\text{outgoing flow}}.$$

The **value** of flow f is:

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) = \text{total flow exiting the source.}$$

Example (network above, flow with value $|f| = 4$):



Flow f/c, value $|f| = 4$

14.6 Residual Network

Definition 14.5: Residual Network G_f

Given a flow f in $G = (V, E)$, the **residual capacity** is:

$$c_f(u,v) = \begin{cases} c(u,v) - f(u,v) & \text{if } (u,v) \in E \quad \text{(remaining capacity)} \\ f(v,u) & \text{if } (v,u) \in E \quad \text{(cancellable flow)} \\ 0 & \text{otherwise.} \end{cases}$$

The **residual network** is $G_f = (V, E_f)$ where $E_f = \{(u,v) \in V \times V : c_f(u,v) > 0\}$.

Key point 14.1: Role of reversed residual edges

The reversed edges (v,u) in G_f represent the possibility of *reducing* the flow on (u,v), which is equivalent to rerouting it elsewhere. It is this mechanism that allows the algorithm to correct bad initial choices.

14.7 Ford-Fulkerson Method

Algorithm 19 Ford-Fulkerson-Method(G, s, t)

1: Initialize $f(u,v) \leftarrow 0$ for all (u,v)
2: **while** there exists an **augmenting path** p from s to t in G_f **do**
3: **Augment** the flow f along p
4: **end while**
5: **return** f

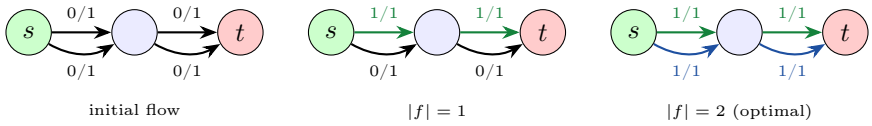
An **augmenting path** is a simple path from s to t in the residual network G_f. We send a flow equal to the **minimum residual capacity** on this path:

$$c_f(p) = \min_{(u,v) \in p} c_f(u,v).$$

For each edge $(u,v) \in p$, the flow is updated by: $f(u,v) \leftarrow f(u,v) + c_f(p) - c_f(v,u)$ (taking into account reversed edges).

14.7.1 Step-by-step example

Diamond graph with a central edge of capacity 1 (the two paths of capacity 1 each):



Case where reversed edges are necessary. Without them, a bad first choice (path passing through the central edge) would stall at $|f| = 1$ instead of 2. The residual network allows "undoing" this choice via a back edge.

14.8 Lecture Summary

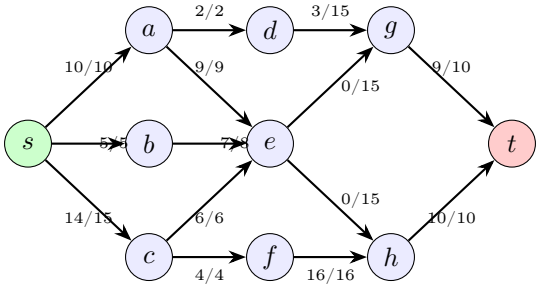
Key Points

- **SCC:** maximal set of mutually reachable vertices.
- G^{SCC} **is a DAG.**
- **Kosaraju:** 2 DFS (G then G^T in decreasing order of $u.f$). Complexity $\Theta(V + E)$.
- **Flow Network:** capacity constraint $0 \leq f(u, v) \leq c(u, v)$ and flow conservation at every vertex $\neq s, t$.
- **Residual Network G_f:** encodes what can still be sent (or cancelled) on each edge.
- **Ford-Fulkerson:** augment along $s \rightsquigarrow t$ paths in G_f until exhaustion.

15 Lecture 15 : Flows and cut - Max-Flow Min-Cut

15.1 Full Example of Ford-Fulkerson

To illustrate the method, here is the evolution of the flow on a network with integer capacities until the optimum $|f| = 29$:



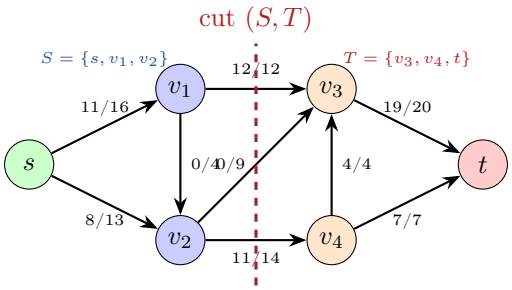
Optimal flow: $|f| = 29$ — no augmenting path in G_f

Cuts in Flow Networks

15.2 Definition of a Cut

Definition 15.1: Cut

A **cut** of the flow network $G = (V, E)$ is a partition of V into two sets S and $T = V \setminus S$ such that $s \in S$ and $t \in T$.



15.3 Net Flow and Capacity of a Cut

Definition 15.2: Net Flow and Capacity of a Cut

The **net flow** through the cut (S,T) is:

$$f(S,T) = \underbrace{\sum_{u\in S,\, v\in T} f(u,v)}_{\text{flow leaving } S} - \underbrace{\sum_{u\in S,\, v\in T} f(v,u)}_{\text{flow entering } S}.$$

The **capacity** of the cut is:

$$c(S,T) = \sum_{u\in S,\, v\in T} c(u,v).$$

Example (cut $S = \{s, v_1, v_2\}$): the net flow is $12+11-4 = 19$ and the capacity is $12+14 = 26$.

15.4 Net Flow is Always Equal to the Flow Value

Theorem 15.1: Net Flow = Flow Value

For any cut (S,T),

$$|f| = f(S,T).$$

Proof. By induction on $|S|$.

Base Case ($S = \{s\}$): $f(\{s\},T) = \sum_v f(s,v) - \sum_v f(v,s) = |f|$ by definition of the flow value.

Inductive Step ($S = S' \cup \{w\}$): when adding w to S', the edges between w and S' move from the boundary to the interior of S. The net flow variation is exactly flow entering w − flow leaving w = 0 by flow conservation ($w \neq s,t$). Therefore $f(S,T) = f(S',T)$ and the value remains $|f|$. □

Immediate Consequence: since $f(u,v) \geq 0$ and $f(u,v) \leq c(u,v)$,

$$|f| = f(S,T) \leq \sum_{u\in S, v\in T} f(u,v) \leq \sum_{u\in S, v\in T} c(u,v) = c(S,T).$$

Thus **every flow is bounded by the capacity of any cut**, which implies:

$$\text{max-flow} \leq \text{min-cut}.$$

15.5 Max-Flow Min-Cut Theorem

Theorem 15.2: Max-Flow Min-Cut

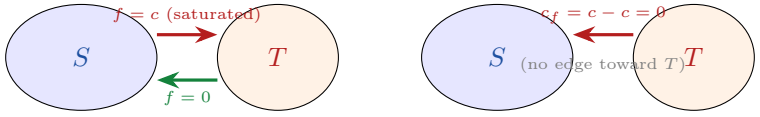
Let $G = (V, E)$ be a flow network with source s, sink t and capacities c. The following three assertions are equivalent:

1. f is a **maximum** flow.
2. G_f has **no augmenting path**.
3. $|f| = c(S, T)$ for a **minimum** cut (S, T).

Proof. (1) $\Rightarrow$ (2)**:** Suppose by contradiction that G_f has an augmenting path p. The Ford-Fulkerson method would augment f along p, increasing the flow value, which contradicts that f is maximum.

(2) $\Rightarrow$ (3)**:** Let $S = \{v \in V : \text{there exists a path from } s \text{ to } v \text{ in } G_f\}$ and $T = V \setminus S$. By assumption, $t \notin S$, so (S, T) is a valid cut.

- **Every edge** (u, v) **leaving** S **in** G has $f(u, v) = c(u, v)$ (saturated). Otherwise $c_f(u, v) = c(u, v) - f(u, v) > 0$ and v would be reachable from s in G_f, contradicting $v \notin S$.
- **Every edge** (v, u) **entering** S **from** T **in** G has $f(v, u) = 0$. Otherwise $c_f(u, v) = f(v, u) > 0$ and v would be reachable from s in G_f.



graph G *residual network G_f*

Therefore:

$$|f| = f(S, T) = \sum_{\substack{u \in S \\ v \in T}} f(u, v) - \sum_{\substack{u \in S \\ v \in T}} f(v, u) = \sum_{\substack{u \in S \\ v \in T}} c(u, v) - 0 = c(S, T).$$

(3) $\Rightarrow$ (1)**:** Since $|f| \leq c(S, T)$ for any cut, if $|f| = c(S, T)$ for a minimum cut, the flow cannot be improved. Thus f is maximum. □

15.6 Illustration on the Example

In the 7-node network (optimal flow $|f| = 29$), the minimum cut identified by $S = \{s\} \cup \{\text{nodes reachable from } s \text{ in } G_f\}$ satisfies $c(S, T) = 29$: the edges of the cut are exactly saturated, and no augmenting path exists in G_f.

15.7 Lecture Summary

Key Points – Flows and Cuts

- **Cut** (S, T): partition $V = S \cup T$ with $s \in S$, $t \in T$.
- **Net Flow** $f(S, T) = \sum_{S \to T} f - \sum_{T \to S} f$.
- **Capacity** $c(S, T) = \sum_{u \in S, v \in T} c(u, v)$.
- **Fundamental Identity:** $|f| = f(S, T)$ for any cut, hence $|f| \leq c(S, T)$.
- **Max-Flow Min-Cut Theorem:**

$$\max |f| = \min c(S, T).$$

  Three equivalent conditions: max flow $\Leftrightarrow$ no augmenting path in G_f $\Leftrightarrow$ flow equals min-cut capacity.
- **Ford-Fulkerson returns an optimal flow** because it stops precisely when condition (2) is verified.

16 Lecture 16 : Ford-Fulkerson complexity and Applications

Complexity of the Ford-Fulkerson Method

16.1 Upper Bound (Integer Capacities)

Complexity 16.1: Ford-Fulkerson — Naive Bound

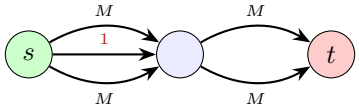
Assuming integer capacities:

- Finding an augmenting path in G_f takes $O(E)$ (using BFS or DFS).
- In each iteration, the flow value increases by at least 1.
- **Total Complexity:** $O(E \cdot |f_{\max}|)$ where $|f_{\max}|$ is the maximum flow value.

This bound is correct but can be very poor if the capacities are large.

16.2 Problematic Case

Consider the diamond graph with a central edge of capacity 1 and capacities $M = 2^{100}$ on the other edges:



If Ford-Fulkerson alternately chooses the top path and the bottom path by passing through the central edge (capacity 1), the flow increases by 1 each time. It would take $2M = 2^{101}$ iterations to reach the optimum $|f_{\max}| = 2M$.

Key point 16.1: Irrational Capacities

If capacities are irrational, the Ford-Fulkerson method may **never terminate**: the sequence of flows may converge to a sub-optimal value.

16.3 Variants with Good Complexity

There are two augmenting path choices that guarantee efficient termination for integer capacities:

Augmenting Path Choice	Number of Iterations	Implementation
Shortest Path (BFS)	$\leq \frac{1}{2} \cdot \lvert E \rvert \cdot \lvert V \rvert$	BFS
Fattest Path	$\leq \lvert E \rvert \cdot \log(\lvert E \rvert \cdot U)$	Modified Dijkstra

where U is the maximum flow value.

Shortest Path (Edmonds-Karp algorithm): we choose an augmenting path with the *fewest edges* possible (BFS) at each iteration. The total number of iterations is bounded by $O(|V| \cdot |E|)$, which gives a global complexity of $O(|V| \cdot |E|^2)$.

Fattest Path: we choose the augmenting path whose bottleneck capacity ($c_f(p) = \min_{(u,v)\in p} c_f(u,v)$) is *maximal*. This choice eliminates the problematic case described above.

Maximum Flow Applications

16.4 Bipartite Matching

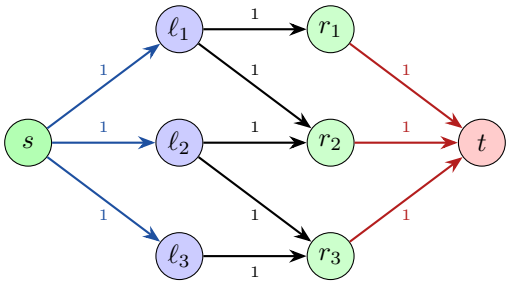
16.4.1 Problem

N students apply for M job offers ($M \geq N$). Each student receives several offers. Can we find a matching that assigns each student to exactly one job for which they were accepted?

Definition 16.1: Bipartite Graph and Matching

A **bipartite graph** $G = (L \cup R, E)$ has two disjoint vertex sets L (students) and R (jobs); all edges go from L to R. A **matching** is a subset of edges such that each vertex is incident to at most one edge of the matching.

16.4.2 Reduction to Maximum Flow



All edges have capacity 1

The construction is:

- Add source s connected to each student (capacity 1).
- Add sink t connected from each job (capacity 1).
- Direct student→job edges (capacity 1).
- Run Ford-Fulkerson.

Theorem 16.1: Max Flow = Max Matching

The value of the maximum flow in this network is equal to the size of the maximum matching in the bipartite graph.

Proof. **Matching ⇒ Flow.** Given a matching of size k, we construct a flow of value k by setting 1 on the matching edges and on the corresponding s-student and job-t edges.

Flow ⇒ Matching. Since capacities are integers, Ford-Fulkerson produces an integer flow (0 or 1 on each edge). Flow conservation implies: each student receives at most 1 unit of flow from s, and each job sends at most 1 unit to t. The student→job edges with flow 1 thus form a matching of size $|f|$. □

16.5 Edge-Disjoint Paths

16.5.1 Problem

In winter, some roads may be closed. How many distinct routes (sharing no edges) exist between Lausanne (s) and Geneva airport (t)? What is the minimum number of roads that must be closed to block all traffic from s to t?

Definition 16.2: Edge-Disjoint Paths

Two paths are **edge-disjoint** if they share no edge. The **maximum number of edge-disjoint paths** from s to t is the largest set of pairwise edge-disjoint $s \rightsquigarrow t$ paths.

16.5.2 Reduction to Maximum Flow

We build a flow network from the road graph:

- Assign a capacity of 1 to each road (in each direction).
- Remove anti-parallel edges via the dummy vertex gadget v' (seen in Lecture 14).
- Run Ford-Fulkerson.

Flow/Cut Duality for Edge-Disjoint Paths

$$\text{max-flow} = \text{maximum number of edge-disjoint paths}$$

$$\text{min-cut} = \text{minimum number of roads to close to isolate } s \text{ from } t$$

The minimum cut identifies exactly the minimal set of roads whose closure blocks every path from Lausanne to Geneva.

16.6 Lecture Summary

Key Points

- **Naive Complexity:** $O(E \cdot |f_{\max}|)$ — can be exponential if capacities are large or irrational.
- **Edmonds-Karp (BFS):** $\leq \frac{1}{2}|E| \cdot |V|$ iterations, thus $O(|V| \cdot |E|^2)$ total.
- **Fattest Path:** $\leq |E| \cdot \log(|E| \cdot U)$ iterations.
- **Bipartite Matching:** reduction to capacity-1 flow network; max flow = max matching (integrality guaranteed by Ford-Fulkerson).
- **Edge-Disjoint Paths:** capacity 1 on each edge; max flow = number of edge-disjoint paths; min cut = min number of edges to remove.

17 Lecture 17: Disjoint set and minimum spanning trees

Disjoint-Set Data Structures (Union-Find)

17.1 Introduction

Definition 17.1: Disjoint-Set Data Structure

A **disjoint-set data structure** (also known as **union-find**) maintains a collection $\mathcal{S} = \{S_1, S_2, \ldots, S_k\}$ of disjoint dynamic sets. Each set is identified by a **representative**, which is some member of the set. The representative doesn't matter as long as querying the representative twice without modifying the set returns the same answer.

The union-find data structure supports three fundamental operations:

- **Make-Set**(x): Create a new set $S_i = \{x\}$ and add S_i to $\mathcal{S}$.
- **Union**(x, y): If $x \in S_x$ and $y \in S_y$, replace S_x and S_y with $S_x \cup S_y$ in $\mathcal{S}$. The representative of the new set is any member in $S_x \cup S_y$ (often the representative of one of S_x or S_y). This operation destroys S_x and S_y since sets must remain disjoint.
- **Find**(x): Return the representative of the set containing x.

17.2 Application: Connected Components

A classic application of union-find is computing connected components in an undirected graph $G = (V, E)$. Two vertices u and v are in the same connected component if and only if there is a path between them. Connected components partition vertices into equivalence classes.

The algorithm proceeds as follows:

- Initialize: For each vertex $v \in V$, call Make-Set(v).
- For each edge $(u, v) \in E$: if Find$(u) \neq$ Find(v), call Union(u, v).

This requires $|V|$ Make-Set operations, at most $2|E|$ Find operations, and at most $|V| - 1$ Union operations, for a total of at most $|V| + 3|E|$ operations.

17.3 Linked List Representation

One implementation represents each set as a singly linked list:

- Each set object maintains a pointer to the head (the representative) and the tail.
- Each list element contains the set member, a pointer to the next element, and a pointer back to the set object.

Operations:

- Make-Set(x): Create a singleton list in time $\Theta(1)$.
- Find(x): Follow the pointer to the set object, then to the representative in $\Theta(1)$.
- Union(x, y): Append y's list onto the end of x's list using x's tail pointer. Update the set pointer for every element in y's list. This can take $\Theta(n)$ time in the worst case.

Weighted-Union Heuristic: Always append the smaller list to the larger list (break ties arbitrarily).

Theorem 17.1: Linked List with Weighted-Union

With the weighted-union heuristic, a sequence of m operations (Make-Set, Union, Find) on n elements takes $O(m + n \log n)$ time.

Proof. Make-Set and Find still take constant time. The key observation is: how many times can each element's representative pointer be updated? Each time an element's pointer is updated, it must be in the smaller set being appended to a larger set. Thus the size of the set containing that element at least doubles. Since the maximum set size is n, each element's pointer can be updated at most $\log n$ times. With n elements and at most n unions, the total cost of all pointer updates is $O(n \log n)$. □

For the connected components problem with $|V|$ elements and $\leq |V| + 3|E|$ operations, the total running time is $O(|V| \log |V| + |E|)$.

17.4 Disjoint-Set Forest

A more efficient implementation represents each set as a rooted tree:

- Each node x has a parent pointer $x.p$.
- The root of each tree is the representative of the set.
- Each node also maintains a rank x.rank (an upper bound on the height).

Operations:

Algorithm 20 Make-Set and Find-Set

```
1: procedure MAKE-SET(x)
2:     x.p ← x
3:     x.rank ← 0
4: end procedure
1: procedure FIND-SET(x)
2:     if x ≠ x.p then
3:         x.p ← FIND-SET(x.p)                    // Path compression
4:     end if
5:     return x.p
6: end procedure
```

Algorithm 21 Union by Rank

```
1: procedure UNION(x, y)
2:     LINK(Find-Set(x), Find-Set(y))
3: end procedure

4: procedure LINK(x, y)
5:     if x.rank > y.rank then
6:         y.p ← x
7:     else
8:         x.p ← y
9:         if x.rank = y.rank then
10:            y.rank ← y.rank + 1
11:        end if
12:    end if
13: end procedure
```

Union by rank: Always attach the tree with smaller rank under the root of the tree with larger rank.

Path compression: During Find-Set(x), make each node on the path from x to the root point directly to the root.

Complexity 17.1: Forest with Union-by-Rank and Path Compression

If we use both union by rank and path compression, a sequence of m operations (Make-Set, Union, Find) on n elements takes $O(m \cdot \alpha(n))$ time, where $\alpha(n)$ is the **inverse Ackermann function**.

Key point 17.1: Inverse Ackermann Function

The function $\alpha(n)$ is an extremely slowly growing function. For all practical purposes, $\alpha(n) \leq 4$:

n	$\alpha(n)$
0 to 2	0
3	1
4 to 7	2
8 to 2047	3
2048 to $A(4,2)$	4

where $A(4,2)$ is a number with more than 10^{19000} digits. Thus, for the connected components problem, the running time is $O((|V|+|E|)\alpha(|V|)) \approx O(|V|+|E|)$, which is essentially linear.

Minimum Spanning Trees

17.5 Problem Definition

Definition 17.2: Minimum Spanning Tree (MST)

Input: An undirected, connected graph $G = (V, E)$ with a weight function $w : E \to \mathbb{R}$ that assigns a weight $w(u, v)$ to each edge $(u, v) \in E$.
Output: A spanning tree $T \subseteq E$ of minimum total weight $w(T) = \sum_{(u,v)\in T} w(u, v)$.
A **spanning tree** is an acyclic subset of edges that connects all vertices. It has exactly $|V| - 1$ edges.

17.6 Applications

Communication Networks: A multinational company wants to lease communication lines between its various locations. Each pair of locations has a cost for leasing a direct line. The company wants to connect all locations with minimum total cost. The solution is the minimum spanning tree of the graph where vertices are locations and edge weights are leasing costs.

17.7 Generic Greedy Approach

Both Kruskal's and Prim's algorithms are based on a generic greedy strategy. The key theoretical tool is the **cut property**.

Definition 17.3: Cut

A **cut** $(S, V \setminus S)$ is a partition of the vertices into two disjoint sets S and $V \setminus S$. An edge (u, v) **crosses** the cut if one endpoint is in S and the other is in $V \setminus S$.

Theorem 17.2: Cut Property (Safe Edge Theorem)

Consider a cut $(S, V \setminus S)$ and let:
- T be a tree on S which is part of some MST
- e be a crossing edge of minimum weight

Then there exists an MST of G containing both e and T.

Proof. If e is already in the MST, we are done. Otherwise, add e to the MST. This creates a cycle with at least one other crossing edge f (since the MST was already spanning). We have $w(f) \geq w(e)$ since e is a minimum weight crossing edge. Replace f by e in the MST. The total weight doesn't increase: $w(\text{new MST}) = w(\text{old MST}) - w(f) + w(e) \leq w(\text{old MST})$. Thus the new tree is also an MST, and it contains both T and e. □

17.8 Prim's Algorithm

Idea: Start with an arbitrary vertex v and set $T = \{v\}$. Greedily grow the tree T: at each step, add to T a minimum-weight crossing edge with respect to the cut $(T, V \setminus T)$.

Correctness: At each iteration, by the cut property, the minimum crossing edge is safe to add. The algorithm terminates when T is a spanning tree.

Implementation Challenge: How do we find the minimum crossing edge at every iteration?

- **Naive approach:** Check all outgoing edges. This requires $O(|E|)$ comparisons at each iteration, giving $O(|E| \cdot |V|)$ total time.
- **Efficient approach:** For every vertex $w \notin T$, maintain $\text{dist}(w)$, the weight of the lightest edge from w to any vertex in T. When a new vertex u is added to T, check whether neighbors of u decrease their distance to T. Use a min-priority queue to extract the vertex with minimum distance.

Complexity 17.2: Prim's Algorithm

With a binary min-heap, Prim's algorithm runs in $O(|E| \log |V|)$ time:
- $|V|$ Extract-Min operations: $O(|V| \log |V|)$
- At most $|E|$ Decrease-Key operations: $O(|E| \log |V|)$

With a Fibonacci heap, the time improves to $O(|E| + |V| \log |V|)$.

17.9 Kruskal's Algorithm

Idea: Start with an empty forest T. Sort all edges by weight in non-decreasing order. Greedily add the cheapest edge that does not create a cycle.

Algorithm 22 Kruskal's MST Algorithm

```
1: procedure MST-KRUSKAL(G, w)
2:     A ← ∅
3:     for each vertex v ∈ V do
4:         MAKE-SET(v)                                  // Initialize disjoint sets
5:     end for
6:     Sort edges of E by weight in non-decreasing order
7:     for each edge (u, v) ∈ E in sorted order do
8:         if FIND-SET(u) ≠ FIND-SET(v) then
9:             A ← A ∪ {(u, v)}                         // Add edge to MST
10:            UNION(u, v)
11:        end if
12:    end for
13:    return A
14: end procedure
```

Correctness: The algorithm maintains the invariant that T is always a sub-forest of some MST. When we add edge (u, v), the endpoints are in different connected components. Consider the cut $(S, V \setminus S)$ where S is the component containing u. Since (u, v) is the cheapest remaining edge and it crosses this cut, by the cut property, it is safe to add.

Implementation Challenge: We need to check whether adding edge (u, v) creates a cycle. This is equivalent to checking whether u and v are already in the same connected component. This is exactly what the union-find data structure provides.

Complexity 17.3: Kruskal's Algorithm

Analysis of Kruskal's algorithm:

- Initialize A: $O(1)$
- First for loop: $|V|$ Make-Set operations
- Sort edges: $O(|E| \log |E|)$
- Second for loop: $O(|E|)$ Find-Set and Union operations
- Total union-find operations: $O((|V| + |E|)\alpha(|V|))$

Total time: $O((|V| + |E|)\alpha(|V|)) + O(|E| \log |E|) = O(|E| \log |E|) = O(|E| \log |V|)$ (since $|E| \leq |V|^2$, we have $\log |E| = O(\log |V|)$).

If edges are already sorted, the time becomes $O(|E|\alpha(|V|))$, which is almost linear.

17.10 Lecture Summary

Key Points

- **Union-Find Data Structure:** maintains disjoint sets with operations Make-Set, Union, and Find. Essential for tracking connected components.
- **Linked list implementation:** $O(m + n \log n)$ time for m operations on n elements with weighted-union heuristic.
- **Forest implementation:** $O(m \cdot \alpha(n))$ time with union-by-rank and path compression, where $\alpha(n)$ is the inverse Ackermann function (practically constant).
- **MST Problem:** Find a spanning tree of minimum total weight in a weighted undirected graph.
- **Cut Property:** The minimum weight crossing edge is safe to add to any partial MST.
- **Prim's Algorithm:** Grow the tree from a single vertex by repeatedly adding the minimum crossing edge. Time: $O(|E| \log |V|)$ with binary heap.
- **Kruskal's Algorithm:** Sort edges and add the cheapest edge that doesn't create a cycle using union-find. Time: $O(|E| \log |V|)$ (sorting dominates).
- Both algorithms are greedy and produce correct MSTs by the cut property.

18 Lecture 18:

The Shortest Path Problem

18.1 Problem Definition

Definition 18.1: Shortest Path Problem

Input: A directed graph $G = (V, E)$ with edge weights $w : E \to \mathbb{R}$ that assigns a weight $w(u, v)$ to each edge $(u, v) \in E$.
The **weight of a path** $p = \langle v_0, v_1, \ldots, v_k \rangle$ is:

$$w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

The **shortest-path weight** from u to v is:

$$\delta(u, v) = \begin{cases} \min\{w(p) : u \rightsquigarrow v\} & \text{if a path from } u \text{ to } v \text{ exists} \\ \infty & \text{otherwise} \end{cases}$$

A **shortest path** from u to v is any path p with weight $w(p) = \delta(u, v)$.

18.2 Problem Variants

There are several variants of the shortest path problem:

- **Single-source:** Find shortest paths from a source vertex s to every other vertex.
- **Single-destination:** Find shortest paths to a given destination vertex from all vertices. Can be solved by single-source by reversing edge directions.
- **Single-pair:** Find the shortest path from u to v. No algorithm known that is better in the worst case than solving single-source.

- **All-pairs:** Find shortest paths from u to v for all pairs of vertices. Can be solved by running single-source for each vertex, but better algorithms exist.

We will focus on the **single-source shortest path problem**.

18.3 Negative Weights and Cycles

We allow negative edge weights in the graph.

Key point 18.1: Negative-Weight Cycles

If there is a negative-weight cycle reachable from the source, then shortest paths are not well-defined: we can traverse the cycle repeatedly to get paths of arbitrarily negative weight.
Some algorithms (like Dijkstra's) only work when all edge weights are non-negative. Others (like Bellman-Ford) can handle negative weights and detect negative cycles.

Application: Currency exchange arbitrage. Model currencies as vertices and exchange rates as edge weights (using logarithms to convert products to sums). A negative cycle indicates an arbitrage opportunity.

18.4 Shortest Path Representation

For each vertex $v \in V$, we maintain:

- $v.d$: the current **shortest-path estimate** (upper bound on $\delta(s, v)$)
- $v.\pi$: the **predecessor** of v in the shortest path from s

Initially, $v.d = \infty$ for all $v \neq s$, and $s.d = 0$. The predecessor $v.\pi = \text{NIL}$ for all vertices.

Bellman-Ford Algorithm

18.5 The Relax Operation

The key operation in shortest path algorithms is **relaxation**.

Algorithm 23 Relax Operation

```
1: procedure RELAX(u, v, w)
2:     if v.d > u.d + w(u, v) then
3:         v.d ← u.d + w(u, v)          // Improve estimate
4:         v.π ← u                      // Update predecessor
5:     end if
6: end procedure
```

Intuition: Can we improve the shortest path estimate for v by going through u and taking edge (u, v)? If $u.d + w(u, v) < v.d$, then we have found a better path to v.

18.6 Algorithm Description

The Bellman-Ford algorithm works for graphs with negative weights and can detect negative cycles.

Algorithm 24 Bellman-Ford Algorithm

```
1: procedure BELLMAN-FORD(G, w, s)
2:     Initialize: s.d ← 0, v.d ← ∞ for all v ≠ s, v.π ← NIL for all v
3:     for i = 1 to |V| − 1 do
4:         for each edge (u, v) ∈ E do
5:             RELAX(u, v, w)
6:         end for
7:     end for
8:     for each edge (u, v) ∈ E do                    // Check for negative cycles
9:         if v.d > u.d + w(u, v) then
10:            return false                            // Negative cycle exists
11:        end if
12:    end for
13:    return true
14: end procedure
```

Idea: Iteratively relax all edges. After i iterations, the algorithm has found shortest paths with at most i edges. Since any simple shortest path has at most $|V| - 1$ edges, $|V| - 1$ iterations suffice.

18.7 Correctness

Theorem 18.1: Bellman-Ford Correctness

If the graph $G = (V, E)$ contains no negative-weight cycles reachable from the source s, then after $|V| - 1$ iterations of relaxing all edges, we have $v.d = \delta(s, v)$ for all vertices $v \in V$.

Proof Sketch. Consider any vertex v and let $p = \langle v_0, v_1, \ldots, v_k \rangle$ be a shortest path from $s = v_0$ to $v = v_k$. Since there are no negative cycles, we can assume p is simple, so $k \leq |V| - 1$.

Initially, $v_0.d = 0 = \delta(s, v_0)$. After i iterations, we have $v_i.d = \delta(s, v_i)$ for all i (by induction). After relaxing edge (v_{i-1}, v_i) in iteration i, we have $v_i.d = v_{i-1}.d + w(v_{i-1}, v_i) = \delta(s, v_i)$. After $k \leq |V| - 1$ iterations, $v_k.d = \delta(s, v_k)$. □

18.8 Detecting Negative Cycles

Theorem 18.2: Negative Cycle Detection

There is a negative-weight cycle reachable from the source s if and only if after running $|V| - 1$ iterations of Bellman-Ford, at least one edge (u, v) can still be relaxed (i.e., $v.d > u.d + w(u, v)$).

Proof. ($\Rightarrow$) If a negative cycle is reachable, then shortest paths are not well-defined, and some $v.d$ values will continue to decrease.

($\Leftarrow$) If no negative cycle exists, the correctness theorem guarantees $v.d = \delta(s, v)$ after $|V| - 1$ iterations. By the triangle inequality, $\delta(s, v) \leq \delta(s, u) + w(u, v)$ for all edges, so $v.d \leq u.d + w(u, v)$ and no edge can be relaxed. □

18.9 Complexity Analysis

Complexity 18.1: Bellman-Ford Algorithm

The Bellman-Ford algorithm runs in $O(|V| \cdot |E|)$ time:
- Initialization: $O(|V|)$
- Main loop: $|V| - 1$ iterations, each relaxing all $|E|$ edges: $O(|V| \cdot |E|)$
- Negative cycle check: $O(|E|)$

Total: $O(|V| \cdot |E|)$.

Dijkstra's Algorithm

18.10 Algorithm for Non-Negative Weights

Dijkstra's algorithm is a faster algorithm for the single-source shortest path problem when all edge weights are non-negative.

Idea: Similar to Prim's algorithm for MST. Maintain a set S of vertices whose shortest-path distances from the source are already known. Greedily grow S: at each step, add the vertex $v \notin S$ with minimum shortest-path estimate $v.d$.

Algorithm 25 Dijkstra's Algorithm

```
1:  procedure DIJKSTRA(G, w, s)
2:      Initialize: s.d ← 0, v.d ← ∞ for all v ≠ s, v.π ← NIL for all v
3:      S ← ∅
4:      Q ← V                                   // Min-priority queue keyed by v.d
5:      while Q ≠ ∅ do
6:          u ← EXTRACT-MIN(Q)
7:          S ← S ∪ {u}
8:          for each vertex v ∈ Adj[u] do
9:              RELAX(u, v, w)
10:             if v.d decreased then
11:                 DECREASE-KEY(Q, v, v.d)
12:             end if
13:         end for
14:     end while
15: end procedure
```

18.11 Correctness

Theorem 18.3: Dijkstra's Correctness

If all edge weights are non-negative, Dijkstra's algorithm correctly computes shortest paths from the source s to all vertices.

Proof Sketch. **Loop invariant:** At the start of each iteration of the while loop, for all vertices $v \in S$, we have $v.d = \delta(s, v)$.

Maintenance: Let u be the vertex added to S. We claim $u.d = \delta(s, u)$. Suppose not, and let y be the first vertex along a shorter path to u that is not in S. Let x be y's predecessor in S. Since we chose u and not y, we have $u.d \leq y.d$. But the path $s \rightsquigarrow x \to y$ is a prefix of a shortest path

to u, so $\delta(s, y) \leq \delta(s, u)$. Since weights are non-negative, $y.d = \delta(s, y) \leq \delta(s, u) \leq u.d \leq y.d$, so $u.d = \delta(s, u)$. □

Key point 18.2: Why Non-Negative Weights?

Dijkstra's algorithm fails with negative weights because once a vertex is added to S, we assume its shortest path is final. A negative edge could later provide a shorter path, violating this assumption.

18.12 Implementation and Complexity

Like Prim's algorithm, Dijkstra's performance depends on the priority queue implementation.

Complexity 18.2: Dijkstra's Algorithm

With a binary min-heap, Dijkstra's algorithm runs in $O(|E| \log |V|)$ time:
- Initialization: $O(|V|)$
- While loop: $|V|$ Extract-Min operations: $O(|V| \log |V|)$
- At most $|E|$ Decrease-Key operations: $O(|E| \log |V|)$

Total: $O(|E| \log |V|)$.
With a Fibonacci heap, the time improves to $O(|V| \log |V| + |E|)$.

Comparison with Bellman-Ford:

- Bellman-Ford: $O(|V| \cdot |E|)$, handles negative weights, detects negative cycles
- Dijkstra: $O(|E| \log |V|)$, requires non-negative weights only

18.13 Lecture Summary

Key Points

- **Shortest Path Problem:** Find minimum-weight paths from a source vertex to all other vertices in a weighted directed graph.
- **Relaxation:** Key operation that improves shortest-path estimates by checking if going through an intermediate vertex yields a shorter path.
- **Bellman-Ford Algorithm:** Handles negative edge weights and detects negative cycles. Time: $O(|V| \cdot |E|)$.
- **Correctness:** After $|V| - 1$ iterations of relaxing all edges, shortest paths are found (if no negative cycles exist).
- **Negative Cycle Detection:** If any edge can still be relaxed after $|V| - 1$ iterations, a negative cycle exists.
- **Dijkstra's Algorithm:** Greedy approach for non-negative weights. Uses priority queue like Prim's MST algorithm. Time: $O(|E| \log |V|)$ with binary heap.
- **Correctness:** Maintains invariant that once a vertex is processed, its shortest-path distance is final. Requires non-negative weights.
- Choose Bellman-Ford for negative weights or cycle detection; choose Dijkstra for better performance when all weights are non-negative.

19 Lecture 19: Shortest Paths and Problem Solving

Kruskal's Algorithm (Recap)

19.1 Algorithm Description

Kruskal's algorithm builds a Minimum Spanning Tree (MST) by starting from an empty forest T and greedily maintaining it:

- At each step, add the cheapest edge that does not create a cycle.
- Repeat until all vertices are connected.

Claim: T is always a sub-forest of some MST (proof by induction on the number of edges/components in T).

19.2 Implementation with Union-Find

In each iteration we need to check whether the cheapest edge creates a cycle. This is equivalent to checking whether its two endpoints belong to the same connected component $\Rightarrow$ use the **disjoint-set (union-find)** data structure.

- Initially, each vertex is its own singleton set.
- When edge (u, v) is added to T, perform UNION of the two components.
- To test for a cycle, compare FIND-SET(u) and FIND-SET(v).

19.3 Complexity Analysis

Complexity 19.1: Kruskal's Algorithm

- Initialize A: $O(1)$
- First for-loop: $|V|$ MAKE-SET operations
- Sort E: $O(|E| \log |E|)$
- Second for-loop: $O(|E|)$ FIND-SET and UNION operations
- Total: $O\big((|V| + |E|)\,\alpha(|V|)\big) + O(|E| \log |E|) = O(|E| \log |E|) = O(|E| \log |V|)$

Note: If edges are already sorted, the time reduces to $O(|E|\,\alpha(|V|))$, which is almost linear.

Problem Solving — MST Applications

Key point 19.1: Problem 1 — Minimum Feedback Edge Set

A **feedback edge set** of a graph G is a subset $F \subseteq E$ such that every cycle in G contains at least one edge in F. Equivalently, removing all edges in F makes G acyclic.
Goal: Compute the minimum-weight feedback edge set.
Key insight: $F = E \setminus T$, where T is a maximum spanning tree of G. Equivalently, take all edges *not* in the MST: the MST spans the graph without cycles, so the remaining edges form a minimal set that covers all cycles.

Key point 19.2: Problem 2 — MST Cut and Cycle Properties

Let $G = (V, E)$ be a connected graph with distinct edge weights.

1. **Cut property:** For any partition of V into two subsets, the minimum-weight edge crossing the cut is in every MST.

2. **Cycle property:** The maximum-weight edge in any cycle of G is *not* in the MST.

The Shortest Path Problem

19.4 Definition

Definition 19.1: Shortest Path Problem

Input: A directed graph $G = (V, E)$ with edge weights $w : E \to \mathbb{R}$.
The **weight of a path** $p = \langle v_0, v_1, \ldots, v_k \rangle$ is:

$$w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

The **shortest-path weight** from u to v is:

$$\delta(u, v) = \begin{cases} \min\{w(p) : u \rightsquigarrow v\} & \text{if a path from } u \text{ to } v \text{ exists} \\ \infty & \text{otherwise} \end{cases}$$

A **shortest path** from u to v is any path p with $w(p) = \delta(u, v)$.

19.5 Problem Variants

- **Single-source:** Find shortest paths from a source s to every other vertex.
- **Single-destination:** Find shortest paths to a given destination from all vertices. Can be solved by single-source after reversing all edge directions.
- **Single-pair:** Find the shortest path from u to v. No algorithm known that improves over single-source in the worst case.
- **All-pairs:** Find shortest paths for every pair (u, v). Can be solved by running single-source for each vertex; better algorithms also exist.

Negative Weights and Applications

19.6 Negative-Weight Edges

We allow negative edge weights in the graph.

Key point 19.3: Negative-Weight Cycles

If a negative-weight cycle is reachable from the source, shortest paths are undefined: we can traverse the cycle repeatedly to obtain paths of arbitrarily negative weight (length $-\infty$).

- Some algorithms (e.g. Dijkstra's) only work with non-negative weights.
- Bellman-Ford handles negative weights and can detect negative cycles.

19.7 Why Negative Weights?

Application: Buying and selling currency (arbitrage detection). Model currencies as vertices and log-exchange rates as edge weights. A negative cycle in this graph corresponds to an arbitrage opportunity (a sequence of trades that yields a net profit).

Bellman-Ford Algorithm

19.8 Setup and the Relax Operation

For each vertex $v \in V$, maintain:

- $\ell(v)$: current upper estimate of the shortest-path length from s to v.
- $\pi(v)$: predecessor of v on the current best path from s.

Initialization: $\ell(s) = 0$, $\ell(v) = \infty$ for $v \neq s$, $\pi(v) = \text{NIL}$.

The key subroutine is **relaxation**:

Algorithm 26 Relax Operation

```
1: procedure RELAX(u, v, w)
2:     if ℓ(v) > ℓ(u) + w(u, v) then
3:         ℓ(v) ← ℓ(u) + w(u, v)                    // Improve estimate
4:         π(v) ← u                                  // Update predecessor
5:     end if
6: end procedure
```

Intuition: Can we shorten the path to v by routing through u? If $\ell(u) + w(u, v) < \ell(v)$, then yes.

19.9 Algorithm

Algorithm 27 Bellman-Ford Algorithm

```
1: procedure BELLMAN-FORD(G, w, s)
2:     INIT-SINGLE-SOURCE(G, s)          // ℓ(s) ← 0; ℓ(v) ← ∞; π(v) ← NIL
3:     for i = 1 to |V| − 1 do
4:         for each edge (u, v) ∈ E do
5:             RELAX(u, v, w)
6:         end for
7:     end for
8:     for each edge (u, v) ∈ E do                // Check for negative cycles
9:         if ℓ(v) > ℓ(u) + w(u, v) then
10:            return false                       // Negative cycle detected
11:        end if
12:    end for
13:    return true
14: end procedure
```

Idea: After i iterations, the algorithm has found all shortest paths using at most i edges. Since any simple path has at most $|V| - 1$ edges, $|V| - 1$ iterations suffice.

19.10 Correctness

Key point 19.4: Key note

Bellman-Ford is only guaranteed to return correct shortest paths if **no negative cycles** are reachable from the source. It can, however, be used to *detect* such cycles.

Theorem 19.1: Optimal Substructure

If $\langle s, v_1, \ldots, v_k, v_{k+1} \rangle$ is a shortest path from s to v_{k+1}, then $\langle s, v_1, \ldots, v_k \rangle$ is a shortest path from s to v_k.
Proof: Suppose toward contradiction that there exists a shorter path p' from s to v_k. Then $p' \to (v_k, v_{k+1})$ would be shorter than the assumed shortest path to v_{k+1}, a contradiction.

Theorem 19.2: Bellman-Ford Correctness

Invariant: After the i-th iteration, $\ell(v)$ is at most the length of the shortest path from s to v using at most i edges.
Proof by induction:

- *Base case:* After 0 iterations, $\ell(s) = 0$ and $\ell(v) = \infty$ for $v \neq s$. Trivially correct.
- *Inductive step:* Consider any shortest path from s to v_{k+1} using at most i edges. By optimal substructure, the prefix to v_k is the shortest path using at most $i-1$ edges. By the induction hypothesis, $\ell(v_k) \leq w(p)$ after the previous iteration. After relaxing (v_k, v_{k+1}) in the i-th iteration, $\ell(v_{k+1}) \leq \ell(v_k) + w(v_k, v_{k+1}) \leq$ (length of shortest path using at most i edges).

If there are no negative cycles reachable from s, then for any vertex v there is a shortest path using at most $n-1$ edges (a path with $\geq n$ edges must contain a cycle of non-negative weight that can be removed). Hence Bellman-Ford returns the correct answer after $n-1$ iterations.

19.11 Detecting Negative Cycles

Theorem 19.3: Negative Cycle Detection

There is a negative-weight cycle reachable from the source s **if and only if** at least one ℓ-value changes when a further (the n-th) iteration of Bellman-Ford is executed.
Proof ($\Rightarrow$): From the correctness proof, if there are no negative cycles, ℓ-values do not change in the n-th iteration.
Proof ($\Leftarrow$): Suppose no ℓ-value changes in the n-th iteration. Then for all $(u, v) \in E$: $\ell(u) + w(u, v) \geq \ell(v)$. For any cycle $v_0 \to v_1 \to \cdots \to v_{t-1} \to v_0$:

$$\sum_{i=1}^{t} \ell(v_i) \;\leq\; \sum_{i=1}^{t} \big(\ell(v_{i-1}) + w(v_{i-1}, v_i)\big) \;=\; \sum_{i=1}^{t} \ell(v_{i-1}) + \sum_{i=1}^{t} w(v_{i-1}, v_i)$$

The two ℓ-sums cancel (cyclic), giving $0 \leq \sum_{i=1}^{t} w(v_{i-1}, v_i)$, so the cycle is non-negative. Hence no negative cycle is reachable.

19.12 Complexity Analysis

Complexity 19.2: Bellman-Ford Algorithm

- Init-Single-Source: $\Theta(|V|)$
- Nested for-loops: $|V| - 1$ iterations, each relaxing all $|E|$ edges: $\Theta(|E| \cdot |V|)$
- Final negative-cycle check: $\Theta(|E|)$

Total: $\Theta(|E| \cdot |V|)$

19.13 Final Comments on Bellman-Ford

- Can detect negative cycles: run for n iterations and look for cycles in the "shortest-path tree" — they correspond to negative cycles.
- Easy to implement in **distributed** settings: each vertex repeatedly queries its neighbours for the best path. This makes it useful for routing protocols and dynamic networks.

19.14 Problem Solving — Ski Resort Sanity Check

Key point 19.5: Exam-style Problem

Lindsey Vonn's assistant has mapped a ski resort as a directed weighted graph $G = (V, E)$ where vertices are stations and edge weights are altitude differences (positive for uphill, negative for downhill).

Sanity check: Starting from any station A, the total altitude change when returning to A should always be 0 — i.e. no cycle should have a strictly negative or positive total weight.

Algorithm: Run Bellman-Ford from any source vertex.

- If the algorithm reports a negative cycle, the map is inconsistent.
- Since the graph models altitude, a positive cycle would also be invalid; check both directions (or equivalently, also check for negative cycles in the reversed graph).

Running time: $O(|V| \cdot |E|)$.

Dijkstra's Algorithm

19.15 Algorithm for Non-Negative Weights

Dijkstra's algorithm is a faster alternative when **all edge weights are non-negative**.

Key ideas:

- Only works with non-negative weights.
- Greedy and faster than Bellman-Ford.
- Similar to Prim's algorithm for MST (essentially a weighted BFS).

Approach: Maintain a set S of vertices whose shortest-path distances are finalised. Greedily grow S: at each step, add the vertex $v \notin S$ that is *closest* to s, i.e.

$$\arg\min_{v \notin S} \min_{u \in S} \big(\ell(u) + w(u, v)\big)$$

Algorithm 28 Dijkstra's Algorithm

```
1: procedure DIJKSTRA(G, w, s)
2:     INIT-SINGLE-SOURCE(G, s)
3:     S ← ∅
4:     Q ← V                                   // Min-priority queue keyed by ℓ(v)
5:     while Q ≠ ∅ do
6:         u ← EXTRACT-MIN(Q)
7:         S ← S ∪ {u}
8:         for each vertex v ∈ Adj[u] do
9:             RELAX(u, v, w)
10:            if ℓ(v) decreased then
11:                DECREASE-KEY(Q, v, ℓ(v))
12:            end if
13:        end for
14:    end while
15: end procedure
```

19.16 Correctness

Theorem 19.4: Dijkstra's Correctness

Loop invariant: At the start of each iteration, for all $v \in S$, $\ell(v) = \delta(s, v)$ (the distance estimate is exact).
Proof sketch: Let u be the vertex being added to S. Suppose $\ell(u) > \delta(s, u)$. Let y be the first vertex on a true shortest path to u that is not yet in S, and x its predecessor in S. Since u was chosen over y, we have $\ell(u) \leq \ell(y)$. But $\ell(y) = \delta(s, y) \leq \delta(s, u) \leq \ell(u)$, so $\ell(u) = \delta(s, u)$, contradicting the assumption.
Why non-negative weights are required: Once u is added to S, its distance is considered final. A negative-weight edge could later yield a shorter path, violating the invariant.

19.17 Implementation and Complexity

Like Prim's, Dijkstra's performance depends on the priority queue implementation.

Complexity 19.3: Dijkstra's Algorithm

- With a **binary min-heap**: each operation takes $O(\log |V|)$, giving total $O(|E| \log |V|)$.
- With a **Fibonacci heap**: $O(|V| \log |V| + |E|)$.

19.18 Comparison: Bellman-Ford vs. Dijkstra

Algorithm	Time complexity	Handles negative weights?
Bellman-Ford	$O(\|V\| \cdot \|E\|)$	Yes (also detects negative cycles)
Dijkstra	$O(\|E\| \log \|V\|)$	No (requires non-negative weights)

19.19 Problem Solving — Dijkstra Correctness Proof

Key point 19.6: Exercise

Show that Dijkstra's algorithm is correct by proving the following loop invariant:

"At the start of each iteration, for all $v \in S$, the distance $\ell(v)$ from s to v is equal to the shortest-path distance $\delta(s, v)$."

The proof proceeds by induction on $|S|$ as sketched in the correctness theorem above.

19.20 Lecture Summary

Key Points

- **Kruskal's Algorithm:** Greedy MST algorithm using union-find. Time: $O(|E| \log |V|)$; almost linear if edges are pre-sorted.
- **Shortest Path Problem:** Find minimum-weight paths from a source to all other vertices in a weighted directed graph.
- **Relaxation:** Core operation: improve $\ell(v)$ if routing through u is cheaper.
- **Bellman-Ford:** Handles negative weights; detects negative cycles. Time: $\Theta(|V| \cdot |E|)$. Correct after $|V| - 1$ iterations when no negative cycles exist.
- **Negative Cycle Detection:** A negative cycle is reachable iff some ℓ-value decreases in the n-th iteration.
- **Dijkstra's Algorithm:** Greedy, faster alternative for non-negative weights. Uses a min-priority queue (analogous to Prim's MST). Time: $O(|E| \log |V|)$ with a binary heap.
- **Correctness of Dijkstra:** Maintains the invariant that each finalised vertex has its exact shortest-path distance. Fails with negative weights.
- **Choice:** Use Bellman-Ford when negative weights or cycle detection are needed; use Dijkstra for better performance on non-negative graphs.

20 Lecture 20-21: Probabilistic Analysis and Hash Tables

Probabilistic Analysis and Randomized Algorithms

20.1 Motivation

Worst-case analysis can be overly pessimistic. Two complementary approaches exist:

- **Average-case analysis:** Assume the input is drawn from some probability distribution and analyse the expected behaviour.
- **Amortized analysis:** Analyse the average cost per operation over a sequence of operations (no probability involved).
- **Randomization:** The algorithm itself introduces randomness (e.g. choosing the pivot in quicksort at random) to avoid worst-case inputs and to foil adversarial users who might try to craft bad inputs.
- Randomization is also **necessary** in cryptography. Obtaining randomness raises its own questions (extractors, pseudorandom generators).

Probabilistic Analysis: The Hiring Problem

20.2 Problem Setup

The NY Knicks interview n basketball candidates one by one. They adopt the strategy: *hire a candidate if and only if they are taller than every previously seen candidate.*

Question: How many candidates do we (temporarily) hire?

Worst case: If candidates arrive in increasing order of height, we hire all n. This happens in only one specific permutation of $n!$, so it is very unlikely.

20.3 Indicator Random Variables

Definition 20.1: Indicator Random Variable

Given a sample space Ω and an event A, the **indicator random variable** of A is:

$$I\{A\} = \begin{cases} 1 & \text{if } A \text{ occurs} \\ 0 & \text{otherwise} \end{cases}$$

Lemma: For any event A, let $X_A = I\{A\}$. Then $\mathbf{E}[X_A] = \Pr[A]$.
Proof: $\mathbf{E}[X_A] = 1 \cdot \Pr\{A\} + 0 \cdot \Pr\{\bar{A}\} = \Pr\{A\}$.

Key tool — Linearity of Expectation: For any random variables X and Y and constants a, b:

$$\mathbf{E}[aX + bY] = a\,\mathbf{E}[X] + b\,\mathbf{E}[Y]$$

This holds *even if X and Y are dependent.*

Example — n coin flips: Let $X_i = I\{\text{flip } i \text{ is heads}\}$. The total number of heads $X = X_1 + \cdots + X_n$. By linearity:

$$\mathbf{E}[X] = \sum_{i=1}^{n} \mathbf{E}[X_i] = \sum_{i=1}^{n} \Pr\{H\} = \frac{n}{2}$$

20.4 Analysis of the Hiring Problem

Assume candidates arrive in a uniformly random order. Define indicator variables:

$$X_i = I\{\text{candidate } i \text{ is hired}\}, \quad X = X_1 + X_2 + \cdots + X_n$$

Key observation: Candidate i is hired if and only if they are the tallest among the first i candidates. Since the order is random, any of the first i is equally likely to be the tallest, so:

$$\Pr\{\text{candidate } i \text{ is hired}\} = \frac{1}{i}$$

By the linearity of expectation:

$$\mathbf{E}[X] = \sum_{i=1}^{n} \mathbf{E}[X_i] = \sum_{i=1}^{n} \Pr\{\text{candidate } i \text{ is hired}\} = \sum_{i=1}^{n} \frac{1}{i} = H_n = \ln n + O(1)$$

where H_n is the n-th harmonic number.

Key point 20.1: Hiring Problem Conclusions

- The **expected number of hires** is $\Theta(\ln n)$, much better than worst case n.
- $\Pr\{\text{hire exactly 1 candidate}\} = 1/n$ (tallest arrives first).
- $\Pr\{\text{hire all } n \text{ candidates}\} = 1/n!$ (strictly increasing order).
- Examples: $n = 6 \Rightarrow 2.45$, $n = 100 \Rightarrow 5.19$, $n = 10000 \Rightarrow 9.79$.

20.5 Randomized Algorithm

Instead of assuming a random arrival order, the algorithm itself **shuffles the candidates into a random order** before beginning the interviews. This guards against adversarial inputs and guarantees the same expected $O(\ln n)$ hires regardless of the original order.

Key point 20.2: Generating an Unbiased Bit

Given a biased coin RANDOM returning 1 with probability p and 0 with probability $1-p$ (unknown $p \neq 0, 1$):

1. Draw a pair (a, b) where $a = \text{RANDOM}()$ and $b = \text{RANDOM}()$.
2. If $a \neq b$, return a (since $\Pr[a = 1, b = 0] = \Pr[a = 0, b = 1] = p(1-p)$).
3. Otherwise, draw a new pair.

This produces an unbiased bit because $\Pr[\text{return } 1] = \Pr[\text{return } 0] = p(1-p)$.

The Birthday Paradox

20.6 Statement

Question: How many people must be in a room so that two of them share a birthday with probability at least 50%? (Assume each of the 365 days is equally likely.)

- Trivially: 366 people guarantee a collision (pigeonhole).
- Surprisingly: only **23 people** are needed for a 50% probability, and **57 people** suffice for 99%.

20.7 Birthday Lemma

Theorem 20.1: Birthday Lemma

If $q > 1.78\sqrt{|M|}$, then the probability that a function chosen uniformly at random $f : \{1, 2, \ldots, q\} \to M$ is injective is at most $\frac{1}{2}$.

Proof. Let $m = |M|$. The probability that f is injective (no two inputs share an output) is:

$$\frac{m}{m} \cdot \frac{m-1}{m} \cdot \frac{m-2}{m} \cdots \frac{m-(q-1)}{m}$$

Since $e^{-x} > 1 - x$, this product is less than:

$$e^0 \cdot e^{-1/m} \cdot e^{-2/m} \cdots e^{-(q-1)/m} = e^{-q(q-1)/(2m)}$$

This is at most $\frac{1}{2}$ when:

$$q \geq \frac{1 + \sqrt{1 + 8\ln 2}}{2}\sqrt{m} \approx 1.78\sqrt{m}$$

Implication for hashing: To store K keys in a table of size m without any collision with good probability, we need $m > K^2$. For a library with 10,000 books, this would require an array of size 10^8 — completely impractical.

Hash Functions and Tables

20.8 Direct-Address Tables

A **direct-address table** T has one slot for each possible key. An element with key k is stored in $T[k]$.

Algorithm 29 Direct-Address Table Operations

```
1: procedure DIRECT-ADDRESS-SEARCH(T, k)
2:     return T[k]
3: end procedure
4: procedure DIRECT-ADDRESS-INSERT(T, x)
5:     T[x.key] ← x
6: end procedure
7: procedure DIRECT-ADDRESS-DELETE(T, x)
8:     T[x.key] ← NIL
9: end procedure
```

Positives	Negatives
$O(1)$ per operation	Space $O(\|U\|)$ where U is the universe of keys
Simple implementation	Most applications store a tiny fraction of $\|U\|$
	Wish for space proportional to items stored

20.9 Hash Tables

Definition 20.2: Hash Table

A **hash table** stores K keys using space $\Theta(K)$. An element with key k is stored in slot $h(k)$, where $h : U \to \{0, 1, \ldots, m-1\}$ is the **hash function** and m is the table size (much smaller than $|U|$).
Simple uniform hashing assumption: h maps each new key equally likely to any of the m slots, independently of all other keys. The **load factor** $\alpha = n/m$ is the average number of elements per slot (where n is the number of keys stored).

Desired properties of a hash function:

- Efficiently computable.
- Distributes keys uniformly across slots (to minimise collisions).
- Deterministic: $h(k)$ is always the same value for the same k.

20.10 Collisions and Chaining

A **collision** occurs when two keys $k_i \neq k_j$ satisfy $h(k_i) = h(k_j)$. By the Birthday Lemma, collisions are unavoidable unless $m > K^2$, which is impractical.

Collision resolution by chaining: Each slot $T[j]$ holds a doubly linked list of all elements that hash to j.

Algorithm 30 Chained Hash Table Operations

```
procedure CHAINED-HASH-SEARCH(T, k)
    Search for an element with key k in list T[h(k)]
end procedure
procedure CHAINED-HASH-INSERT(T, x)
    Insert x at the head of list T[h(x.key)]
end procedure
procedure CHAINED-HASH-DELETE(T, x)
    Delete x from the list T[h(x.key)]
end procedure
```

- **Insertion:** $O(1)$ (insert at head).
- **Deletion:** $O(1)$ since the list is doubly linked and we have a pointer to the element.
- **Space:** $O(m + K)$.

20.11 Running Time Analysis

Let n_j be the length of list $T[j]$. Then $n = n_0 + n_1 + \cdots + n_{m-1}$ and under simple uniform hashing $\mathbf{E}[n_j] = \alpha = n/m$.

Theorem 20.2: Unsuccessful Search

Under simple uniform hashing, an unsuccessful search takes expected time $\Theta(1 + \alpha)$.
Proof: Any key not in the table is equally likely to hash to any slot. An unsuccessful search for key k traverses the entire list $T[h(k)]$, which has expected length $\mathbf{E}[n_{h(k)}] = \alpha$. Adding the $O(1)$ cost of computing $h(k)$, the total expected time is $\Theta(1 + \alpha)$.

Theorem 20.3: Successful Search

Under simple uniform hashing, a successful search takes expected time $\Theta(1 + \alpha)$.
Proof: Let x_i be the i-th element inserted and $k_i = \text{key}[x_i]$. For all $i < j$, define the indicator variable $X_{ij} = I\{h(k_i) = h(k_j)\}$. Under simple uniform hashing, $\mathbf{E}[X_{ij}] = 1/m$. The expected number of elements examined in a successful search is:

$$\mathbf{E}\left[\frac{1}{n}\sum_{i=1}^{n}\left(1 + \sum_{j=i+1}^{n} X_{ij}\right)\right] = \frac{1}{n}\sum_{i=1}^{n}\left(1 + \sum_{j=i+1}^{n}\mathbf{E}[X_{ij}]\right) = \frac{1}{n}\sum_{i=1}^{n}\left(1 + \sum_{j=i+1}^{n}\frac{1}{m}\right) = \cdots = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}$$

which is $\Theta(1 + \alpha)$.

20.12 Consequence: Constant-Time Operations

Complexity 20.1: Hash Table with Chaining

If we choose the table size $m = \Theta(n)$ (proportional to the number of elements stored), then $\alpha = n/m = \Theta(1)$ and:

- **Insertion:** $O(1)$
- **Deletion:** $O(1)$
- **Search (expected):** $O(1)$
- **Space:** $\Theta(n)$

20.13 Examples of Hash Functions

Designing good hash functions is a large research area that depends on the data distribution. Two basic methods are:

Division method:

$$h(k) = k \bmod m$$

m is often chosen to be a prime not too close to a power of 2, to spread keys evenly.

Multiplicative method:

$$h(k) = \lfloor m \cdot \{Ak\} \rfloor$$

where $\{Ak\}$ denotes the fractional part of Ak. Knuth suggests $A \approx (\sqrt{5} - 1)/2 \approx 0.618$.

20.14 Lecture Summary

Key Points

- **Probabilistic Analysis:** Instead of worst-case, analyse expected performance assuming random input (or random algorithm choices).
- **Indicator Random Variables:** $I\{A\}$ is 1 if A occurs, 0 otherwise. Lemma: $\mathbf{E}[I\{A\}] = \Pr[A]$.
- **Linearity of Expectation:** $\mathbf{E}[aX + bY] = a\mathbf{E}[X] + b\mathbf{E}[Y]$, even for dependent variables. Allows decomposing complex random variables.
- **Hiring Problem:** Expected number of hires is $H_n = \ln n + O(1)$ when candidates arrive in random order.
- **Randomized Algorithms:** The algorithm itself shuffles the input to guarantee good expected performance against any adversary.
- **Birthday Paradox:** Only $\approx 1.78\sqrt{m}$ items are needed before a collision in a universe of size m occurs with constant probability. Collisions in hash tables are unavoidable unless the table is quadratically large.
- **Direct-Address Tables:** $O(1)$ operations but $O(|U|)$ space; impractical for large universes.
- **Hash Tables with Chaining:** $\Theta(K)$ space, $O(1)$ insert/delete, and $\Theta(1 + \alpha)$ expected search time.
- **Load Factor:** $\alpha = n/m$. Choosing $m = \Theta(n)$ gives $\alpha = \Theta(1)$ and all operations in expected $O(1)$ time.
- **Hash Function Design:** Two basic methods are the division method ($h(k) = k \bmod m$) and the multiplicative method. Choice depends on data distribution.

21 Lecture 22: Hash sort and quick sort

Hash Tables: Summary (Recap)

Complexity 21.1: Hash Tables

Hash tables efficiently implement the following operations:

- **Insert:** $O(1)$
- **Delete:** $O(1)$
- **Search:** Expected $O(n/m)$ with a good hash function (i.e. $O(1)$ when $m = \Theta(n)$)

Collisions cannot be avoided without having $m \gg n^2$ (Birthday Lemma). Instead, they are handled using, for example, **chaining**.

Quick Sort

21.1 Overview

- The sorting algorithm of choice in many computer systems.
- Easy to implement.
- Fast in practice (and as we will prove in theory).
- Like merge sort, based on the **divide-and-conquer** paradigm.

21.2 Divide-and-Conquer Structure

To sort the subarray $A[p \ldots r]$:

- **Divide:** Partition $A[p \ldots r]$ into two (possibly empty) subarrays $A[p \ldots q-1]$ and $A[q+1 \ldots r]$, such that every element in the left subarray is $\leq A[q]$ and every element in the right subarray is $\geq A[q]$. The element $A[q]$ is called the **pivot**.
- **Conquer:** Sort the two subarrays by recursive calls to QUICKSORT.
- **Combine:** No work needed — the subarrays are sorted in place.

Algorithm 31 Quick Sort

```
1: procedure QUICKSORT(A, p, r)
2:     if p < r then
3:         q ← PARTITION(A, p, r)
4:         QUICKSORT(A, p, q − 1)
5:         QUICKSORT(A, q + 1, r)
6:     end if
7: end procedure
```

21.3 The Partition Procedure

PARTITION always selects the last element $A[r]$ as the pivot and rearranges the subarray in place.

Algorithm 32 Partition

```
1: procedure PARTITION(A, p, r)
2:     x ← A[r]                                   // Pivot
3:     i ← p − 1
4:     for j = p to r − 1 do
5:         if A[j] ≤ x then
6:             i ← i + 1
7:             exchange A[i] with A[j]
8:         end if
9:     end for
10:    exchange A[i + 1] with A[r]                // Place pivot in correct position
11:    return i + 1
12: end procedure
```

21.4 Correctness of Partition

Theorem 21.1: Loop Invariant of Partition

At the start of each iteration of the **for** loop, for any index k:

1. If $p \le k \le i$, then $A[k] \le x$ (pivot).
2. If $i+1 \le k \le j-1$, then $A[k] > x$.
3. $A[r] = x$ (pivot remains in place until the final swap).

Initialization: Before the loop, both subarrays $A[p \dots i]$ and $A[i+1 \dots j-1]$ are empty, and $A[r] = x$. The invariant holds trivially.

Maintenance: If $A[j] \le x$, increment i and swap $A[i]$ with $A[j]$, then increment j. If $A[j] > x$, only increment j.

Termination: When the loop terminates, $j = r$, so all elements are partitioned into $A[p \dots i] \le x$, $A[i+1 \dots r-1] > x$, and $A[r] = x$. The final two lines swap $A[i+1]$ with $A[r]$, placing the pivot in its correct position $q = i+1$.

21.5 Running Time of Partition

Complexity 21.2: Partition

- The **for** loop runs $n := r - p + 1$ times.
- Each iteration takes $\Theta(1)$.
- Total: $\Theta(n)$ for an array of length n.
- The number of comparisons made is $\approx n$.

Running Time Analysis of Quick Sort

21.6 Worst Case

The worst case occurs when PARTITION always produces a maximally unbalanced split: one subarray of size $n-1$ and one of size 0. This happens when the input is already sorted (ascending or descending) and the pivot is always the smallest or largest element.

The recursion tree has n levels each costing $\Theta(n), \Theta(n-1), \dots, \Theta(1)$:

$$T(n) = T(n-1) + T(0) + \Theta(n) = T(n-1) + \Theta(n) \;\Rightarrow\; T(n) = \Theta(n^2)$$

21.7 Best Case

The best case occurs when PARTITION always produces a perfectly balanced split, with both subarrays of size $n/2$:

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$$

by the Master Theorem.

21.8 Average Case — Intuition

Key point 21.1: Unbalanced splits still give $\Theta(n \log n)$

Even a constant-ratio unbalanced split (e.g. always 9-to-1) gives:

$$T(n) = T\left(\tfrac{9n}{10}\right) + T\left(\tfrac{n}{10}\right) + \Theta(n) = \Theta(n \log n)$$

In practice, splits alternate between good and bad. Alternating best-case and worst-case splits yields the same asymptotic running time $\Theta(n \log n)$ as always splitting evenly,

because the total work at each level of the tree remains $\Theta(n)$.

Randomized Quick Sort

21.9 Motivation and Algorithm

The deterministic worst case occurs on sorted inputs — exactly the inputs that arise most often in practice. To eliminate this vulnerability:

- Instead of always using $A[r]$ as the pivot, **randomly select** a pivot from $A[p \dots r]$ and swap it with $A[r]$ before calling PARTITION.
- This guarantees good expected performance **for any input**, not just random ones.

Key point 21.2: Key distinction

There is a huge difference between:

- *Expected running time over all inputs* (average-case of deterministic algorithm): depends on input distribution, can be manipulated by an adversary.
- *Expected running time for any fixed input* (randomized algorithm): the randomness is internal; no adversary can force worst-case behaviour.

Algorithm 33 Randomized Quick Sort

```
1:  procedure RANDOMIZED-PARTITION(A, p, r)
2:      i ← RANDOM(p, r)                          // Pick a random index in [p, r]
3:      exchange A[r] with A[i]
4:      return PARTITION(A, p, r)
5:  end procedure
6:  procedure RANDOMIZED-QUICKSORT(A, p, r)
7:      if p < r then
8:          q ← RANDOMIZED-PARTITION(A, p, r)
9:          RANDOMIZED-QUICKSORT(A, p, q − 1)
10:         RANDOMIZED-QUICKSORT(A, q + 1, r)
11:     end if
12: end procedure
```

21.10 Expected Running Time Analysis

Theorem 21.2: Expected Running Time of Randomized Quicksort

Randomized QUICKSORT has expected running time $O(n \log n)$ for *any* input of size n.

Proof.

Setup: The dominant cost is partitioning. Each call to PARTITION costs $O(1)$ plus the number of comparisons in the for loop. Since each element is chosen as pivot at most once, PARTITION is called at most n times. Let X be the total number of comparisons across all calls; then the total work is $O(n + X)$.

Counting comparisons with indicator variables: Rename the elements $z_1 < z_2 < \cdots < z_n$ (sorted order). Define the set $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$ and the indicator variable:

$$X_{ij} = I\{z_i \text{ is compared to } z_j\}$$

Since each pair is compared at most once (only when one of them is the pivot), the total

number of comparisons is:

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$$

Applying linearity of expectation:

$$\mathbf{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbf{E}[X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \Pr[z_i \text{ is compared to } z_j]$$

Computing the probability: z_i and z_j are compared if and only if the first element chosen as pivot from Z_{ij} is either z_i or z_j. If any element strictly between them is chosen first, they are separated into different subarrays and never compared. Since the pivot is chosen uniformly at random from Z_{ij}, which has $j - i + 1$ elements:

$$\Pr[z_i \text{ is compared to } z_j] = \frac{2}{j - i + 1}$$

Summing up: Setting $k = j - i$:

$$\mathbf{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n-1} \sum_{k=1}^{n} \frac{2}{k} = \sum_{i=1}^{n-1} O(\log n) = O(n \log n)$$

Therefore the expected running time is $O(n + X) = O(n \log n)$.

21.11 Summary of Quick Sort

Key Points

- **Quick Sort:** Divide-and-conquer sorting algorithm. Pivot partitions the array in place; no merge step needed.
- **Partition:** Runs in $\Theta(n)$. Loop invariant guarantees correct placement of pivot.
- **Worst case:** $\Theta(n^2)$ — occurs when partitioning is always maximally unbalanced (e.g. already sorted input with last-element pivot).
- **Best case:** $\Theta(n \log n)$ — perfectly balanced splits every time.
- **Average case intuition:** Even a constant-ratio split (e.g. 9-to-1) gives $\Theta(n \log n)$. Mixing good and bad splits does not change the asymptotic bound.
- **Randomized Quicksort:** Pick the pivot uniformly at random from the subarray. Guarantees expected $O(n \log n)$ for *any* input, foiling adversarial inputs.
- **Key proof idea:** z_i and z_j are compared iff one of them is the first pivot chosen from Z_{ij}, giving $\Pr[z_i \text{ vs. } z_j] = \frac{2}{j-i+1}$. Summing by linearity of expectation yields $\mathbf{E}[X] = O(n \log n)$.
- **Practical advantages:** In-place, small constants, very efficient in practice.

22 Lecture 24: Online Learning and Experts

22.1 Introduction: Online vs Offline Algorithms

Definition 22.1: Offline Algorithm

An algorithm where the **entire input is known in advance**. The algorithm can plan with full information.

Examples: sorting an array, finding a Minimum Spanning Tree (MST).

Definition 22.2: Online Algorithm

An algorithm where **input arrives one piece at a time**. The algorithm makes **irrevocable decisions** without knowledge of future requests.

Key principle: The algorithm must act before it knows the whole story.

Online decision problems arise in many practical settings:

Setting	Online Decision
Cache eviction	Which page to evict?
Online ads	Which ad to show?
Rent or buy	Keep renting or buy?
Expert advice	Whose advice today?
Hiring/selection	Accept now or wait?

A reasonable decision may look poor after the future is revealed. The relevant question is not "was the algorithm psychic?" but rather: **how badly can it be punished for not being psychic?**

22.2 Measuring Performance

Definition 22.3: Competitive Ratio

Compare the algorithm's cost with the best solution that knew the whole input in advance (the offline optimum):

$$\mathrm{ALG}(I) \leq r \cdot \mathrm{OPT}(I) + c$$

Used when the offline optimum is a meaningful and stable target.

When comparing to the full offline optimum is too ambitious, we use a weaker but still meaningful benchmark.

Definition 22.4: Regret

Compare the algorithm's total loss with the **best single expert in hindsight**. The extra loss incurred by the algorithm is called **regret**:

$$\text{regret} = \text{loss}(\mathrm{ALG}) - \min_i \text{loss}(i)$$

Used when "best possible" is too strong or too unstable as a benchmark.

Key point 22.1: Performance benchmarks

Happiness $\approx$ outcome $-$ expectation, but happiness is also *relative* to the people next to you. Similarly, algorithm performance is judged relative to what was achievable.

22.3 Prediction with Expert Advice

22.3.1 The Expert Setting

Consider the following scenario: every morning, N experts each predict YES or NO (e.g., "will it rain?"). We must make our own prediction before observing the outcome.

The game. For $t = 1, \ldots, T$:

1. Each expert $i \in \{1, \ldots, N\}$ predicts YES or NO.
2. The algorithm predicts YES or NO.
3. The true outcome is revealed.
4. Everyone who predicted incorrectly receives one *mistake*.

Let M_{ALG} denote the number of mistakes of the algorithm, and M_i the number of mistakes of expert i.

Goal: Keep M_{ALG} close to $\min_i M_i$.

22.3.2 Why Compare to the Best Expert?

- We cannot promise to make few mistakes on every sequence (e.g., if all experts say YES and the answer is NO, any algorithm using their advice is in trouble).
- But if one expert was consistently good, we would like to discover this quickly.
- **Regret** asks: is $M_{\text{ALG}} - \min_i M_i$ small?

22.4 Warmup: One Perfect Expert

22.4.1 The Elimination Algorithm

Suppose at least one expert makes *zero* mistakes. We can exploit this with a simple elimination strategy.

Elimination Algorithm

1. Keep the experts who have made **no mistakes** so far.
2. Predict by **majority vote** among the remaining experts.
3. After the outcome is revealed, **delete** every expert who was wrong.

Correctness: The perfect expert is never deleted, so the set of remaining experts is never empty.

22.4.2 Analysis

Theorem 22.1: Perfect Expert Guarantee

If at least one expert makes no mistakes, the Elimination algorithm makes **at most** $\lg N$ **mistakes**.

Proof sketch. Suppose the algorithm makes a mistake. Since it followed the majority of remaining experts, at least half of the remaining experts were wrong — and are now deleted. The pool size decreases:

$$N \to \frac{N}{2} \to \frac{N}{4} \to \cdots$$

After more than $\lg N$ mistakes, fewer than one expert would remain — impossible since the perfect expert survives. □

Example: With $N = 1024$ experts, even a very adversarial sequence causes at most 10 mistakes before the algorithm has essentially identified the perfect expert.

22.4.3 When Nobody is Perfect

A first idea: run Elimination until all experts are deleted, then restart with all experts alive. If the best expert makes M mistakes, there are at most $M + 1$ restarts, giving:

$$M_{\text{ALG}} \leq (M + 1) \lg N$$

This bound is often **too weak**: the overhead grows *multiplicatively* with M. We want an **additive** penalty instead.

22.5 Weighted Majority

22.5.1 Soft Elimination

Instead of deleting an expert after a mistake, we *reduce its influence*:

- If correct today: **keep weight**.
- If wrong today: **halve weight**.
- Wrong often: weight **becomes negligible**.

The algorithm listens to a **weighted majority vote**.

22.5.2 The Algorithm

Weighted Majority Algorithm

Initially set $w_i^{(1)} = 1$ for every expert i. For $t = 1, \ldots, T$:

1. Each expert predicts YES or NO.
2. Predict the answer with **larger total weight**.
3. After the outcome is revealed, update:

$$w_i^{(t+1)} = \begin{cases} w_i^{(t)}/2 & \text{if expert } i \text{ was wrong,} \\ w_i^{(t)} & \text{otherwise.} \end{cases}$$

Weights act as **trust scores**: they only decrease when the expert makes a mistake, so a good expert retains much more weight than a bad one.

22.5.3 Guarantee

Theorem 22.2: Weighted Majority Bound

Let M_{WM} be the number of mistakes of Weighted Majority. For every expert i:

$$M_{\text{WM}} \leq 2.41 \cdot (M_i + \lg N)$$

Therefore:

$$M_{\text{WM}} \leq 2.41 \cdot \left(\min_i M_i + \lg N \right)$$

This is much better than the restarting bound when M_i is large: the overhead is now **additive** in $\lg N$.

22.5.4 Analysis via Potential Function

Define the **potential** (total weight):

$$\Phi^{(t)} = \sum_{i=1}^{N} w_i^{(t)}$$

Upper bound (from algorithm's mistakes): When the algorithm makes a mistake, it followed the weighted majority, so at least half the total weight was on wrong experts. Those weights are halved, giving:

$$\Phi^{(t+1)} \leq \frac{1}{2}\Phi^{(t)} + \frac{1}{2} \cdot \frac{1}{2}\Phi^{(t)} = \frac{3}{4}\Phi^{(t)}$$

After M_{WM} mistakes:

$$\Phi^{(T+1)} \leq N \left(\frac{3}{4}\right)^{M_{\text{WM}}}$$

Lower bound (from a good expert): If expert i makes M_i mistakes, its final weight is $(1/2)^{M_i}$. Since the total weight is at least this:

$$\left(\frac{1}{2}\right)^{M_i} \leq \Phi^{(T+1)}$$

Combining:

$$\left(\frac{1}{2}\right)^{M_i} \leq N \left(\frac{3}{4}\right)^{M_{\text{WM}}}$$

Taking base-2 logarithms and rearranging:

$$M_{\text{WM}} \leq \frac{1}{\lg(4/3)}(M_i + \lg N) \approx 2.41\,(M_i + \lg N)$$

Intuition: Many algorithm mistakes $\Rightarrow$ total weight becomes tiny. A good expert prevents this from happening.

22.5.5 Tuning the Penalty

Halving (multiplying by $1/2$ on a mistake) is a special case. More generally, we can multiply by $(1 - \varepsilon)$ for any $\varepsilon \in (0, 1)$.

Theorem 22.3: Tunable Weighted Majority

With a suitable weighted-majority update using parameter ε:

$$M_{\mathrm{WM}} \leq 2(1+\varepsilon)\, M_i + O\left(\frac{\lg N}{\varepsilon}\right) \quad \text{for every expert } i$$

Tradeoff: Smaller ε means less overreaction to mistakes (more patient), but slower learning.

22.6 Randomized Experts: The Hedge Algorithm

22.6.1 Limitation of Weighted Majority

Weighted Majority is **deterministic**. An adversary who can observe our prediction can always choose the opposite outcome, forcing a mistake every round. In fact, there exist sequences where deterministic Weighted Majority makes *about twice* as many mistakes as the best expert. To improve the constant, we stop committing to one answer deterministically.

22.6.2 Randomized Prediction

Instead of choosing one expert's advice deterministically, we choose a **distribution** $\mathbf{p}^{(t)} = (p_1^{(t)}, \ldots, p_N^{(t)})$ over experts. If expert i incurs loss $m_i^{(t)} \in [-1, 1]$ on day t, our **expected loss** is:

$$\sum_{i=1}^{N} p_i^{(t)} m_i^{(t)} = \mathbf{p}^{(t)} \cdot \mathbf{m}^{(t)}$$

This framework covers binary mistakes, but also richer cost structures: money lost, error magnitude, reward minus cost, etc.

22.6.3 The Hedge Algorithm

Hedge Algorithm (parameter $\varepsilon > 0$)

Maintain weights $w_i^{(t)}$. At time t:

1. Play expert i with probability

$$p_i^{(t)} = \frac{w_i^{(t)}}{\Phi^{(t)}}, \quad \text{where } \Phi^{(t)} = \sum_j w_j^{(t)}$$

2. Observe the loss vector $\mathbf{m}^{(t)}$.
3. Update weights:

$$w_i^{(t+1)} = w_i^{(t)} \cdot e^{-\varepsilon m_i^{(t)}}$$

Low loss $\Rightarrow$ weight increases; high loss $\Rightarrow$ weight decreases. The key distinction from Weighted Majority is the use of the **exponential update**, which handles continuous losses naturally.

22.6.4 Hedge Guarantee

Theorem 22.4: Hedge Regret Bound

If $m_i^{(t)} \in [-1, 1]$ and $\varepsilon \leq 1$, then for every expert i:

$$\sum_{t=1}^{T} \mathbf{p}^{(t)} \cdot \mathbf{m}^{(t)} \leq \sum_{t=1}^{T} m_i^{(t)} + \frac{\ln N}{\varepsilon} + \varepsilon T$$

Choosing $\varepsilon \approx \sqrt{\ln N / T}$ balances the two additive terms and gives:

$$\text{regret} = O\left(\sqrt{T \ln N}\right)$$

The **average regret** vanishes as $T \to \infty$:

$$\frac{\text{regret}}{T} = O\left(\sqrt{\frac{\ln N}{T}}\right) \to 0$$

This is called **no-regret** or **Hannan consistency**.

22.7 Why Multiplicative Weights is Everywhere

The Multiplicative Weights / Hedge framework is a small idea with an enormous footprint across computer science:

- **Boosting** (ML): combine many weak classifiers into a strong one (AdaBoost is a special case).
- **Game theory**: repeated play with Hedge converges to correlated equilibria.
- **Optimization**: fast approximate algorithms for linear programs and semidefinite programs (SDPs).
- **Online learning**: portfolio selection, prediction markets, and adaptive decision making.
- **Algorithm design**: a standard potential-function proof template.

Key point 22.2: The recurring pattern

Multiplicatively reward what worked, but do not completely forget alternatives.

This is the fundamental principle behind multiplicative weights: unlike additive updates that can send weights negative or zero, exponential updates keep all options alive while shifting mass toward better-performing experts.

22.8 Summary

Algorithm	Guarantee	Setting
Elimination	$M_{\text{ALG}} \leq \lg N$	One perfect expert exists
Restart Elimination	$M_{\text{ALG}} \leq (M^* + 1) \lg N$	General, but multiplicative overhead
Weighted Majority	$M_{\text{WM}} \leq 2.41(M^* + \lg N)$	General, deterministic
Hedge	$\text{regret} = O(\sqrt{T \ln N})$	General, randomized, continuous losses

where $M^* = \min_i M_i$ is the number of mistakes of the best expert.

Key takeaways:

- Online algorithms must make irrevocable decisions without seeing the future.
- When comparing to full hindsight is too ambitious, **regret** is the right benchmark.
- The **Elimination algorithm** achieves $\lg N$ mistakes with a perfect expert.
- **Weighted Majority** handles imperfect experts with an additive $O(\lg N)$ overhead, analyzed via a potential function argument.
- **Hedge** uses randomization and exponential updates to achieve sublinear $O(\sqrt{T \ln N})$ regret.

23 Lecture 25 : Online Algorithms and Competitive Analysis

23.1 From Regret to the Hindsight Optimum

The previous lecture measured an online algorithm against the **best single expert in hindsight** (regret). Today we adopt a more demanding yardstick: the **offline optimum** OPT, which sees the entire request sequence and makes the best possible decisions with full knowledge of the future.

Last lecture	$\text{loss}(\text{ALG}) \le \min_i \text{loss}(i) + \text{small regret}$
Today	$\text{cost}_{\text{ALG}}(I) \le r \cdot \text{cost}_{\text{OPT}}(I) + c$

The offline optimum is a very strong benchmark: it is allowed to know the future, while the online algorithm must commit to **irrevocable** decisions as requests arrive.

23.2 Competitive Analysis

Definition 23.1: Competitive Ratio

For a minimization problem, an online algorithm is **r-competitive** if there is a constant c (independent of the length of I) such that for *every* input sequence I,

$$\text{cost}_{\text{ALG}}(I) \;\le\; r \cdot \text{cost}_{\text{OPT}}(I) + c.$$

If $c = 0$ the bound is sometimes called a *strong* competitive ratio. For maximization problems the ratio is written the other way ($\text{ALG} \ge \frac{1}{r}\,\text{OPT}$).

Why this benchmark is useful:

- **Instance-by-instance:** we compare ALG and OPT on the *same* sequence.
- **Distribution-free:** no probabilistic assumption on the future is needed.
- **Robust:** even adversarial request sequences are covered.
- **But possibly pessimistic:** sometimes *no* online algorithm can stay close to OPT.

23.3 Rent or Buy: The Ski Rental Problem

Each ski day we may either **rent** skis for cost 1, or **buy** them once for cost B (and ski free forever after). We do not know in advance how many ski days T there will be. The offline optimum,

knowing T, pays

$$\mathrm{OPT}(T) = \min\{T, B\}.$$

The online decision is simply: *when should we buy?*

Rent-then-Buy

Rent every day until the **total rental cost reaches** B (i.e. for B days). Then buy.

Theorem 23.1: Rent-then-Buy is 2-competitive

For every season length T, $\mathrm{ALG}(T) \le 2\,\mathrm{OPT}(T)$.

Proof. Two cases on T.

- **Case $T \le B$:** the algorithm only ever rents, so $\mathrm{ALG}(T) = T = \mathrm{OPT}(T)$.
- **Case $T > B$:** the algorithm rents for B days (cost B) and then buys (cost B), so $\mathrm{ALG}(T) \le 2B$. The optimum buys immediately, paying $\mathrm{OPT}(T) = B$. Hence $\mathrm{ALG}(T) \le 2B = 2\,\mathrm{OPT}(T)$.

In both cases $\mathrm{ALG}(T) \le 2\,\mathrm{OPT}(T)$. □

No deterministic algorithm beats 2. Any deterministic rule buys on some fixed day x (if skiing lasts long enough). An adversary, knowing x, sets the season length: if $x \le B$ it stops the season exactly when the algorithm buys; if $x > B$ it lets skiing continue until the buy. For large B the ratio is forced arbitrarily close to 2, so rent-then-buy is essentially the best deterministic strategy.

Key point 23.1: Randomization helps

A randomized buy-day achieves expected competitive ratio $\frac{e}{e-1} \approx 1.58$, which is optimal against an oblivious adversary. This $\frac{e}{e-1}$ constant recurs throughout online algorithms.

23.4 Caching (Paging)

A cache holds k pages. Requests $\sigma = (p_1, \ldots, p_T)$ arrive online. A request to a page already in cache is a **hit** (cost 0); otherwise it is a **miss** (cost 1), the page is brought in, and if the cache is full some page must be **evicted**. The cost is the number of misses, and the online decision is *which page to evict.*

23.4.1 The Offline Optimum: Belady's Rule

Theorem 23.2: Farthest-in-the-Future (Belady)

When the entire request sequence is known, the optimal eviction rule is: on a miss, evict the page whose **next request is farthest in the future**.

Intuition: keep pages needed soon, evict pages not needed for a long time. This is *not* an online algorithm — it is the benchmark OPT.

23.4.2 Online Eviction Rules

Rule	Evicts
LRU	the least recently used page
FIFO	the page that entered the cache earliest
LFU	the least frequently used page
LIFO	the page that entered most recently

Some rules are arbitrarily bad. With a cache of size k initially holding $\{1, \dots, k\}$, the stream $k{+}1,\ k,\ k{+}1,\ k,\ \dots$ makes **LIFO** miss on *every* request, while OPT misses once and then keeps both k and $k{+}1$. So LIFO has *unbounded* competitive ratio; a similar construction defeats LFU.

23.4.3 Deterministic Lower Bound

Theorem 23.3: No deterministic rule beats k

Every deterministic online caching algorithm has competitive ratio at least k.

Adversary. Restrict to pages $\{1, \dots, k+1\}$. At each step request the unique page among these that is *not* currently in the algorithm's cache. The online algorithm then misses on *every* request. Meanwhile OPT (farthest-in-the-future) can arrange at least k hits between consecutive misses. Hence the best possible deterministic ratio is $\geq k$.

23.4.4 LRU is k-competitive

Phases. Partition σ into **phases**: a phase is a maximal consecutive block containing at most k distinct pages; a new phase begins exactly when a request would introduce a $(k{+}1)$st distinct page. Let P be the number of phases. For example, with $k = 3$,

$$\underbrace{4, 1, 2, 1}_{\text{phase 1}}\ \underbrace{5, 3, 4, 4}_{\text{phase 2}}\ \underbrace{1, 2, 3}_{\text{phase 3}}.$$

Theorem 23.4: LRU bound

$\mathrm{LRU}(\sigma) \leq k \cdot \mathrm{OPT}(\sigma) + O(k)$.

Upper bound: LRU misses $\leq k$ per phase. A phase has at most k distinct pages, and LRU can miss only on the *first* appearance of each. Once a page has been requested within the phase it is among the $\leq k$ most-recently-used pages, so LRU will not evict it before the phase ends. Hence $\mathrm{LRU}(\sigma) \leq kP$.

Lower bound: OPT misses ≥ 1 per phase. When a new phase starts, a page appears that was not among the k distinct pages of the previous phase. Across the previous phase plus this first new request there are $k + 1$ distinct pages — too many for a size-k cache — so OPT must miss at least once per phase (up to startup): $\mathrm{OPT}(\sigma) \geq P - O(1)$.

Combine. $\mathrm{LRU}(\sigma) \leq kP \leq k\big(\mathrm{OPT}(\sigma) + O(1)\big) = k \cdot \mathrm{OPT}(\sigma) + O(k)$. Together with the lower bound, **LRU is asymptotically optimal** among deterministic online caching algorithms.

Key point 23.2: Randomization helps again

The randomized **marking algorithm** (mark a page when requested; on a miss evict a uniformly random *unmarked* page; when all pages are marked, unmark everything and

start a new phase) achieves

$$\mathbb{E}[\text{misses}] \le 2H_k \cdot \text{OPT} = O(\log k) \cdot \text{OPT},$$

and no randomized algorithm can beat $\Omega(H_k)$. Randomization improves caching from $\Theta(k)$ to $\Theta(\log k)$.

23.5 Online Bipartite Matching

A bipartite graph $G = (L, R, E)$: the offline side L is known in advance; the online vertices R arrive one at a time. When $r \in R$ arrives we see its neighbourhood $N(r) \subseteq L$ and must **immediately** either match r to an unmatched neighbour, or leave it unmatched (irrevocably). The goal is to maximize the number of matched arrivals; $\text{OPT} = |M^\star|$ is a maximum matching of the full graph.

Key point 23.3: Why this model is a classic

Introduced by Karp, Vazirani & Vazirani (STOC 1990), it cleanly separates deterministic ($\frac{1}{2}$) from randomized ($1 - \frac{1}{e}$) guarantees. It is also the core abstraction behind **ad allocation / AdWords**: offline vertices are advertisers (with budgets), online vertices are user impressions, an edge means the ad is eligible, and a matched edge earns revenue. In the unit-budget, unit-value case, ad allocation *is* online bipartite matching.

23.5.1 Deterministic Greedy

Greedy

When r arrives, match it to **any** currently unmatched neighbour in L, if one exists.

Theorem 23.5: Greedy is $\frac{1}{2}$-competitive

For every instance, $|M_{\text{greedy}}| \ge \frac{1}{2}|M^\star|$; equivalently greedy is a deterministic 2-approximation.

Proof (charging). Let M be the greedy matching and $M^\star$ an offline optimum.

Claim: every edge of $M^\star$ touches some edge of M. Take $(l, r) \in M^\star$. If greedy matched r, then (l, r) shares the endpoint r with a greedy edge. If greedy left r unmatched, it was because *all* of r's neighbours — in particular l — were already matched by greedy when r arrived; so (l, r) shares the endpoint l with a greedy edge.

Charge each optimum edge to a greedy edge it touches. A single greedy edge has only two endpoints, so it can be charged by at most two optimum edges, giving $|M^\star| \le 2|M|$. □

Tightness. With $L = \{a, b\}$: the first arrival r_1 is adjacent to both. If the deterministic algorithm matches r_1 to a, the adversary sends r_2 adjacent only to a. The algorithm gets 1 match; OPT matches $r_1 - b$ and $r_2 - a$ for 2 (symmetrically if it had chosen b). No deterministic algorithm guarantees more than $\frac{1}{2}$.

23.5.2 The KVV Ranking Algorithm

The fix is to **break symmetry once**, before any arrivals.

Ranking (Karp–Vazirani–Vazirani)

Pick a uniformly random permutation π of the offline vertices L. When r arrives, match it to the unmatched neighbour with **smallest π-rank**, if one exists.

Ranking is still greedy, but the tie-breaking is *global and randomized*: each offline vertex gets a fixed random priority.

Theorem 23.6: Ranking guarantee (KVV, 1990)

For every fixed online bipartite matching instance,

$$\mathbb{E}\big[|M_{\text{Ranking}}|\big] \;\geq\; \big(1 - \tfrac{1}{e}\big)\,|M^\star|.$$

Moreover, **no** randomized online algorithm can guarantee a ratio better than $1 - \frac{1}{e}$ against an oblivious adversary.

The recurring moral across ski rental, caching, and matching: **randomization can beat the best deterministic guarantee.**

24 Lecture 26 : Secretary and Prophet Inequalities

24.1 Beyond Worst-Case Online Analysis

Competitive analysis assumes the request sequence may be fully **adversarial**. This is robust but can be overly harsh. Two classical ways to give the online algorithm more structure — while keeping decisions **irrevocable** — are:

- items arrive in a **random order** (secretary problem);
- values are drawn from **known distributions** (prophet inequalities).

Model	What we know	Goal
Secretary problem	fixed weights; uniformly random arrival order	pick a maximum-weight item
Prophet inequality	independent values with known distributions	get close to $\mathbb{E}[\max_i X_i]$

Both ask the same question: *how much value is lost because we must decide online?*

24.2 The Secretary Problem

Each candidate j has a fixed unknown weight w_j (ties broken arbitrarily). Candidates arrive in **uniformly random order**; on arrival we see the weight and must **immediately accept or reject**, accepting at most one. The goal is to maximize the probability of selecting the global maximum:

$$\text{maximize} \quad \mathbb{P}\Big[w_{\text{ALG}} = \max_j w_j\Big].$$

Key point 24.1: Why random order is essential

Against a fully adversarial order, any deterministic rule fails: if it accepts some weight, the adversary places a larger one later; if it waits, the adversary places the maximum earlier. Random order is exactly what prevents the maximum from being placed maliciously before or after our stopping rule.

24.2.1 Two Simple Strategies

- **Take the first candidate:** succeeds with probability $\frac{1}{n}$.
- **Sample half, then take the first record:** reject the first $n/2$, then accept the first candidate beating all sampled weights. This succeeds with probability $\geq \frac{1}{4}$, since it suffices that the maximum lies in the second half and the second-largest in the first half: $\frac{1}{2} \cdot \frac{1}{2}$.

24.2.2 The Threshold Strategy

Threshold rule (parameter r)

Reject the first $r-1$ candidates (the **sample**). Then accept the first candidate whose weight beats every previously seen weight (a new **record**).

Choosing r balances two opposing forces: sample enough to *learn*, but do not wait so long that the maximum has already *passed*.

Example ($n = 8$, $r = 4$, higher is better): the stream is $54, 71, 43, 62, 49, 83, 95, 38$ with sample $\{54, 71, 43\}$. The largest sampled weight is 71; the first later record is 83 at position 6, which we accept — *missing* the true maximum 95 at position 7.

24.2.3 Success Probability

Suppose the maximum-weight candidate is at position $i \geq r$. We select it **iff** the largest weight among positions $1, \ldots, i-1$ falls inside the sample $1, \ldots, r-1$ (otherwise an earlier record is accepted first). Since that largest weight is equally likely in any of the first $i-1$ positions,

$$\mathbb{P}[\text{success} \mid \text{max at } i] = \frac{r-1}{i-1}.$$

The position of the maximum is uniform on $\{1, \ldots, n\}$, so

$$\mathbb{P}[\text{success}] = \frac{1}{n} \sum_{i=r}^{n} \frac{r-1}{i-1}.$$

24.2.4 Choosing the Best Threshold

For large n, approximate the sum by an integral. With $\alpha = r/n$,

$$\frac{r-1}{n} \sum_{i=r}^{n} \frac{1}{i-1} \;\approx\; \frac{r}{n} \int_{r}^{n} \frac{1}{x}\, dx = \frac{r}{n} \ln\!\left(\frac{n}{r}\right) = \alpha \ln\!\left(\frac{1}{\alpha}\right).$$

Maximizing $\alpha \ln(1/\alpha)$ over α gives $\alpha = 1/e$, hence:

Theorem 24.1: Optimal secretary guarantee

Sampling the first $\approx n/e$ candidates and then taking the first record selects the maximum-weight candidate with probability

$$\mathbb{P}[\text{success}] \;\approx\; \frac{1}{e} \approx 0.3679,$$

and no algorithm can do better than $1/e$ in the classical secretary model.

The magic is not the exact constant but that a *simple* online rule attains a *constant* probability of selecting the global maximum.

24.3 Prophet Inequalities

There are independent nonnegative random variables $X_1, \dots, X_n$ with **known distributions**. Values are revealed one by one; on seeing X_i we accept (and stop) or reject it forever, accepting at most one. The **prophet** sees all realizations and gets $M = \max_i X_i$; we compare our expected value to

$$\mathbb{E}[M] = \mathbb{E}\Big[\max_i X_i\Big].$$

Key point 24.2: Why distributions matter

Without distributions, two values defeat any rule: with $x_1 = 1$ and x_2 unknown, accepting x_1 lets the adversary set $x_2 = H$ huge, while rejecting it lets the adversary set $x_2 = 0$. Knowing the distributions replaces this impossible worst case by a tractable stochastic one.

A prophet inequality asserts $\mathbb{E}[\mathrm{ALG}] \geq \alpha\, \mathbb{E}[M]$ for some constant α.

24.3.1 A Single Threshold Achieves $\frac{1}{2}$

Threshold rule

Let $z = \mathbb{E}[M]$ and set $\tau = z/2$. Accept the first value X_i with $X_i \geq \tau$.

This is a **posted price**: "stop as soon as the offer is at least τ."

Theorem 24.2: Prophet inequality (Samuel-Cahn)

For independent nonnegative $X_1, \dots, X_n$, the threshold rule with $\tau = \mathbb{E}[\max_i X_i]/2$ satisfies

$$\mathbb{E}[\mathrm{ALG}] \;\geq\; \tfrac{1}{2}\,\mathbb{E}\Big[\max_i X_i\Big].$$

The factor $1/2$ is tight for the classical model.

Proof. Write $M = \max_i X_i$, $z = \mathbb{E}[M]$, $\tau = z/2$, and define

- $q = \mathbb{P}[\text{no value reaches } \tau]$, and $q_i = \mathbb{P}[\text{no value before } i \text{ reaches } \tau]$ (so $q_i \geq q$);
- $b_i = \mathbb{E}\big[(X_i - \tau)^+\big]$, the expected excess of X_i above the threshold.

Lower bound on $\mathbb{E}[\mathrm{ALG}]$ *(two sources of value).* The algorithm earns the threshold τ whenever *some* value reaches it, plus the excess above it whenever it stops on X_i. Using $q_i \geq q$,

$$\mathbb{E}[\mathrm{ALG}] \;\geq\; \tau \cdot \mathbb{P}[\text{some value reaches } \tau] + q \sum_i b_i = \tau(1-q) + q \sum_i b_i.$$

Upper bound on the prophet. For *every* realization, $M \leq \tau + \sum_i (X_i - \tau)^+$ (the maximum is either below τ, contributing $\leq \tau$, or its excess is one of the summands). Taking expectations,

$$z = \mathbb{E}[M] \leq \tau + \sum_i b_i, \qquad \text{so} \qquad \sum_i b_i \geq z - \tau = \tau \quad (\text{since } \tau = z/2).$$

Combine.

$$\mathbb{E}[\mathrm{ALG}] \;\geq\; \tau(1-q) + q \sum_i b_i \;\geq\; \tau(1-q) + q\tau \;=\; \tau \;=\; \frac{z}{2}. \qquad \square$$

The online algorithm gets at least half of what a prophet gets in expectation, using nothing but a single fixed threshold.

24.4 Secretary vs. Prophet

	Secretary	**Prophet**
Information	observed weights	value distributions
Arrival	random order	usually fixed order
Benchmark	max weight	$\mathbb{E}[\max_i X_i]$
Simple rule	sample then record	threshold price
Classic guarantee	$1/e$ success	$1/2$ expected value

Same online tension, different information models — and in both, a **simple threshold / stopping rule competes with hindsight**.